Fusion3M: Community-based Multi-scale Co-evolving Network for Dynamic Graph Representation Learning

Chao Li[a], Qianyu Song[a], Runshuo Liu[a], Zhongying Zhao[b,*] and Qingtian Zeng[a,b]

[a]College of Electronic and Information Engineering, Shandong University of Science and Technology, Qingdao, 266590, China
[b]College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao, 266590, China

ARTICLE INFO

Keywords:
Dynamic Graph Neural Networks
Link Prediction
Community Detection
Information Fusion

ABSTRACT

Dynamic Graph Neural Networks have been demonstrated to be effective in modeling dynamic graph structured data, which enables them to solve tasks such as node classification, link prediction, and popular prediction. Existing research has shown a variety of structures within dynamic graphs, ranging from individual representations characterized by microscopic structure to graph representations characterized by macroscopic structure. However, current works primarily focus on the individual level, while neglecting the group states characterized by mesoscopic structure and the intricate correlations between these different levels. In this work, we present ComCo, a Community-based multi-scale Co-evolving network to establish the intrinsic correlations for the Fusion of Microscopic, Mesoscopic and Macroscopic structure information (Fusion3M). Specifically, we extract interactions and encode historical information to model micro-level representations. Additionally, we incorporate a novel community detection mechanism with graph pooling to model meso-level and macro-level representations. Finally, our framework leverages a hierarchical transformer encoder to achieve multi-scale information fusion by learning the complex relationships between different structural scales. Experimental results on seven real-world datasets and nine baselines demonstrate that our model significantly outperforms the current state-of-the-art techniques on multiple downstream tasks. The source code and data have been made available at https://github.com/SSQiana/ComCo.

1. Introduction

Dynamic graphs can represent many complex relationships in real world, where the nodes represent corresponding entities, and the edges between nodes indicate their interactions. As dynamic graphs evolve, these graphs can realize the addition or removal of nodes, the creation or deletion of edges, as well as the adjustment of weights or the update of attributes. Such dynamic graphs can effectively simulate and reflect the changes of various real-world scenarios, including social networks [1, 2] recommendation systems [3, 4], and transportation networks [5, 6].

Link prediction is a fundamental task in dynamic graphs, with the objective of predicting potential interactions between pairs of nodes in the future. To achieve this goal, current research attempts to extract intricate node relationships. For example, some studies [7, 8, 9] utilize multi-hop neighbors and aggregate their features through mean pooling or attention mechanisms. Other approaches leverage random walks [10, 11, 12] to reveal implicit paths between nodes. Although the aforementioned methods have expanded the neighborhood of nodes, merely averaging or weighted aggregating the features of these node neighbors focuses solely on individual representations, which fails to capture the collective state of groups composed of individuals with similar characteristics. However, it has been proved that [13] individuals and groups in dynamic graphs carry different information about the graph structure: studies at the individual level focus on how nodes establish connections with their surrounding neighbors, while studies at the group level focus on communities which formed by the tight connected individuals with similar characteristics.

Based on the above observations, our central argument is that the temporal dynamics of both individuals and groups are interconnected and mutually influencing. This phenomenon can be demonstrated from two aspects. **On the one hand**, individuals with the same characteristics gathering into groups is one of the most prominent patterns in social networks [14, 15]. Individuals cluster together based on shared characteristics (such as interests or backgrounds), thereby giving rise to communities. The evolving composition of these communities collectively shapes the dynamic network, as depicted in Fig. 1. **On the other hand**, the progression of lower-scale dynamics is typically influenced

*Corresponding author
lichao@sdust.edu.cn (Chao Li); qianasong@163.com (Qianyu Song); liurunshuo@163.com (Runshuo Liu); zzysuin@163.com (Zhongying Zhao); qtzeng@163.com (Qingtian Zeng)

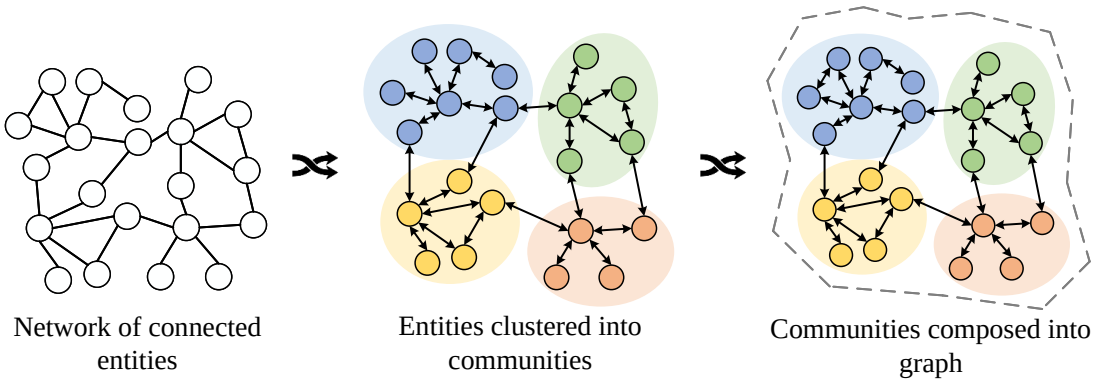


Figure 1: An illustration of the aggregation process from individual to community to network.

by the overarching higher-scale dynamics. Groups are not just collections of individuals, they exert a significant influence and actively shape their members. For instance, the characteristics and behaviors of individuals within the same community can propagate through social networks, leading to increased homogeneity among members over time [16, 17]. Additionally, the interactions between communities can amplify this effect, further influencing the overall structure and evolution of the network.

To fully leverage the multi-scale information on dynamic graphs, we attempt to model representations at different structural scales, and establish their intrinsic connections within continuous-time dynamic graphs, which remains unexplored in the literature. However, this problem is non-trivial and presents the following challenges:

- **Challenge (i): How to develop appropriate strategies to represent different structural scales within the dynamic graphs?** In online platforms, new items and content are constantly refreshed each day. Moreover, the preferences of individual users and groups are continuously evolving. These dynamic changes manifest not only in the diversity of individual behaviors but also in the complexity of interaction patterns within groups, posing significant challenges for representing structures across different scales.

- **Challenge (ii): How to systematically establish the complex dependencies among these structural scales for multi-scale information fusion?** Networks are composed of individuals and their interconnections, with macroscopic temporal dynamics arising from mesoscopic and microscopic dynamics. Conversely, individual behavior is influenced by community dynamics, while mesoscopic community structures are shaped by the network's evolution. Consequently, these relationships are highly entangled across different structural scales.

In response to these challenges, we present a novel and general framework, ComCo, which is designed to capture intrinsic correlations across different structural levels. Specifically, Our framework introduces Fusion3M, a mechanism that facilitates the fusion of microscopic, mesoscopic, and macroscopic structural information. We first encode the representation of each entity and iteratively update these representations based on newly observed interactions. To tackle the challenge that (i) effectively modeling individuals and groups within dynamic graphs is non-trivial, we propose a novel dynamic community detection algorithm that identifies potential node groups with similar semantics. To tackle the challenge that (ii) various relationships are intricately entangled across different structural scales, we propose a Transformer-based encoder that fosters the evolution of lower-scale representations under the influence of higher-scale ones. Additionally, we fuse multi-scale information using a memory update module for various downstream tasks. Our main contributions can be summarized as follows:

- We propose a Community-based multi-scale Co-evolving Network to explicitly characterize microscopic, mesoscopic, and macroscopic structures while facilitating information fusion across the evolving dynamics of different structural scales.

- We propose a novel community detection algorithm to capture group preferences, which dynamically clusters the nodes based on spatial and temporal proximity as well as semantic similarity, and learns a corresponding community level representation.

- We conduct comprehensive experiments on seven datasets and nine baseline models to evaluate the performance of the proposed framework, which cover common graph learning tasks such as link prediction and node classification. The results demonstrate that our work significantly outperforms the existing state-of-the-art baselines.

The remainder of this paper is organized as follows. Section 2 discusses the related work, and Section 3 provides the relevant concepts and formal definitions of dynamic graph. The framework of ComCo and descriptions of each encoding module are presented in Section 4. Section 5 analysis the effectiveness of ComCo with experimental results. Finally, Section 6 concludes this paper and discusses future research directions.

2. Related Work

In this section, we will describe the relevant literature to the research presented in this paper, including dynamic graph representation learning and community detection.

2.1. Dynamic Graph Representation Learning

Existing dynamic graph representation learning methods are mainly divided into two categories based on the granularity of learning time. The first category is Discrete-Time Dynamic Graphs (DTDGs) [18, 19, 20, 21], which divides the original dynamic graph into a series of static graph snapshots. Although simple, it ignores the temporal order of edges within snapshots, which may result in the loss of important temporal information. Our research focuses on the second category, namely Continuous-Time Dynamic Graphs (CTDGs) [22, 11, 8, 9, 7], where the specific time of interaction is directly encoded into edge features to learn time-sensitive node representations. Most CTDGs methods can be further categorized into two subtypes: (i) Recurrent Neural Network (RNN)-based approaches [7, 11, 23, 24], which update temporal node embeddings sequentially upon the arrival of new interactions. These methods leverage RNNs to capture the dependencies across time and ensure that historical interaction information is incorporated effectively. (ii) Self-Attention Mechanism (SAM)-based approaches [22, 9, 25, 26], which dynamically weight and aggregate the embeddings of neighboring nodes for each target node. SAM-based models can focus on the most pertinent neighbors by learning attention scores, thereby facilitating a more adaptable representation of nodes in dynamic graphs.

Although insightful, simply sampling node neighbors in reverse chronological order or by random walks is insufficient to represent the collective state of individuals with similar attributes. In this paper, we attempt to model individuals, groups, and the entire network, and explore the intrinsic correlations between different scales.

2.2. Community Detection

Community detection aims at revealing the complex interactions between clusters in graphs [27, 28]. Incorporating community structure into a deep learning framework poses significant challenges, resulting in substantial computational overhead and intricate assessments of the relationships between communities and their constituent nodes. Existing works have provided various solutions for community detection on graphs Graph Convolutional Networks (GCNs) leverage the aggregation and message-passing abilities of GCNs within node neighborhoods to cluster nodes and discern communities [29, 30, 31]. In addition, Graph Attention Networks (GATs) introduce an attention mechanism that assigns different weights to neighboring nodes based on their relative importance. This allows GATs to enhance the influence of nodes within the same community while reducing the impact of nodes belonging to different communities [32, 33]. Another class of methods involves graph autoencoders, which are designed to learn low-dimensional latent representations of graph data. These methods aim to preserve the original graph structure during the encoding process by minimizing reconstruction loss [34, 35, 36].

As existing communities dissolve and new ones emerge, the task of community detection on such dynamic graphs still remains a significant challenge. In this paper, we propose a novel community detection algorithm for continuous-time dynamic graphs to identify the evolving community structure on dynamic graphs.

3. Preliminaries

Definition 1 (Continuous-Time Dynamic Graph [9]). *A continuous-time dynamic graph is represented as a sequence of interactions arranged in chronological order,* $G = \{\gamma(t_1), \gamma(t_2), \ldots, \gamma(t_n)\}$*, where* $0 \leq t_1 \leq t_2 \leq \ldots \leq t_n$*. Each interaction is characterized by a triple* $\gamma(t) = (u, v, t)$*, with* $u, v \in N$ *denoting the source and target nodes*

involved in the interaction, respectively. The nodes are associated with features $\mathbf{x}_u(t) \in \mathbb{R}^{d_N}$*, and the interactions at time t associate edge features* $\mathbf{e}_{u,v}(t) \in \mathbb{R}^{d_E}$*.*

Definition 2 (Dynamic Community). *Given a dynamic graph G, the set of interactions associated with node u before time t is defined as:* $G_{u,t} = \{(e, t') \mid t' < t, u \in e, (e, t') \in G\}$*, where* $e = \{u, v\}$ *represents an edge, and* (u, v, t') *denotes an interaction tuple. A dynamic community centered at node u is defined as the set of the k most recent temporal neighbors, ordered by time:* $C_u^t = \{(v_0, t_0), (v_1, t_1), \ldots, (v_{k-1}, t_{k-1})\}$*, where* $\{(v_i, t_i) \mid 0 \leq i < k\}$ *, and the elements are ordered such that* $t_0 < t_1 < \cdots < t_{k-1}$*.*

Definition 3 (Temporal-Personalized PageRank (T-PPR) [37]). *Given a dynamic graph G, a source node u with time t, and a target node v with time* τ*, the T-PPR value of* (u, t) *with respect to* (v, τ)*, denoted as* $\pi_{u,t}^{v,\tau}$*, is defined as the probability of a random walk starting at* (u, t) *and terminating at* (v, τ)*.*

Definition 4 (Top-*k* T-PPR Query [37]). *Consider a dynamic graph G. Given a temporal node* (u, t)*, the goal of top-k T-PPR query is to return a dictionary* $\pi^{u,t}$ *of temporal nodes (excluding* (u, t)*) that have the top-k largest T-PPR values with respect to* (u, t)*. For a top-k temporal node* $(v, \tau) \in \pi^{u,t}$*,* $\pi^{u,t}(v, \tau)$ *denotes its T-PPR value.*

Problem Formalization Given a continuous-time dynamic graph G, the representation learning on the dynamic graph aims to learn a mapping function Φ that converts nodes to vectors $\mathbf{h}_u^t \in \mathbb{R}^{3d}$ and $\mathbf{h}_v^t \in \mathbb{R}^{3d}$, where d is the encoding dimension. In this work, the learned node representations will be used for dynamic link prediction and dynamic node classification.

Table 1
Summary of primary notations.

Notation	Description
G	Continuous-time dynamic graph
$\mathbf{s}_C^t$	Memory of the community at time t
$\mathbf{s}_N^t$	Memory of the node at time t
S_u^t	Neighbor sequence of node u at time t
$\mathbf{H}_u^t$	Node-level representation of node u at time t
$\mathbf{Y}_u^t$	Community-level representation of node u at time t
$\mathbf{Z}_u^t$	Graph-level representation of node u at time t
$\pi_{\text{top}}^{u,t}(v, \tau)$	T-PPR value of node (v, τ) with respect to (u, t)
$\pi_{\text{sem}}^{u,t}(u, \tau)$	Attention score of node (v, τ) with respect to (u, t)
$\pi^{u,t}(v, \tau)$	Community member strength of node (v, τ) with respect to (u, t)
α	Termination probability in T-PPR
β	Exponential decay factor in T-PPR
$I(t)$	Community index detected at time t
C_u	Community centered at node u
$N(C_u)$	Members of the community centered at node u
$\mathbf{O}_u(t)$	Community representation centered at node u at time t
$\phi_u(t)$	Community fitness score centered at node u at time t

4. The Proposed Methodology: ComCo

The primary objective of our method is to establish the complex dependencies among microscopic, mesoscopic, and macroscopic structural scales for multi-scale information fusion. To achieve this, we propose a Community-based multi-scale Co-evolving network, which mainly consists of four components: dynamic node representation, dynamic community representation, dynamic graph representation, and the multi-scale co-evolving module. The main architecture of ComCo is shown in Fig. 2. In the following sections, we will elaborate on the details of the four key modules.

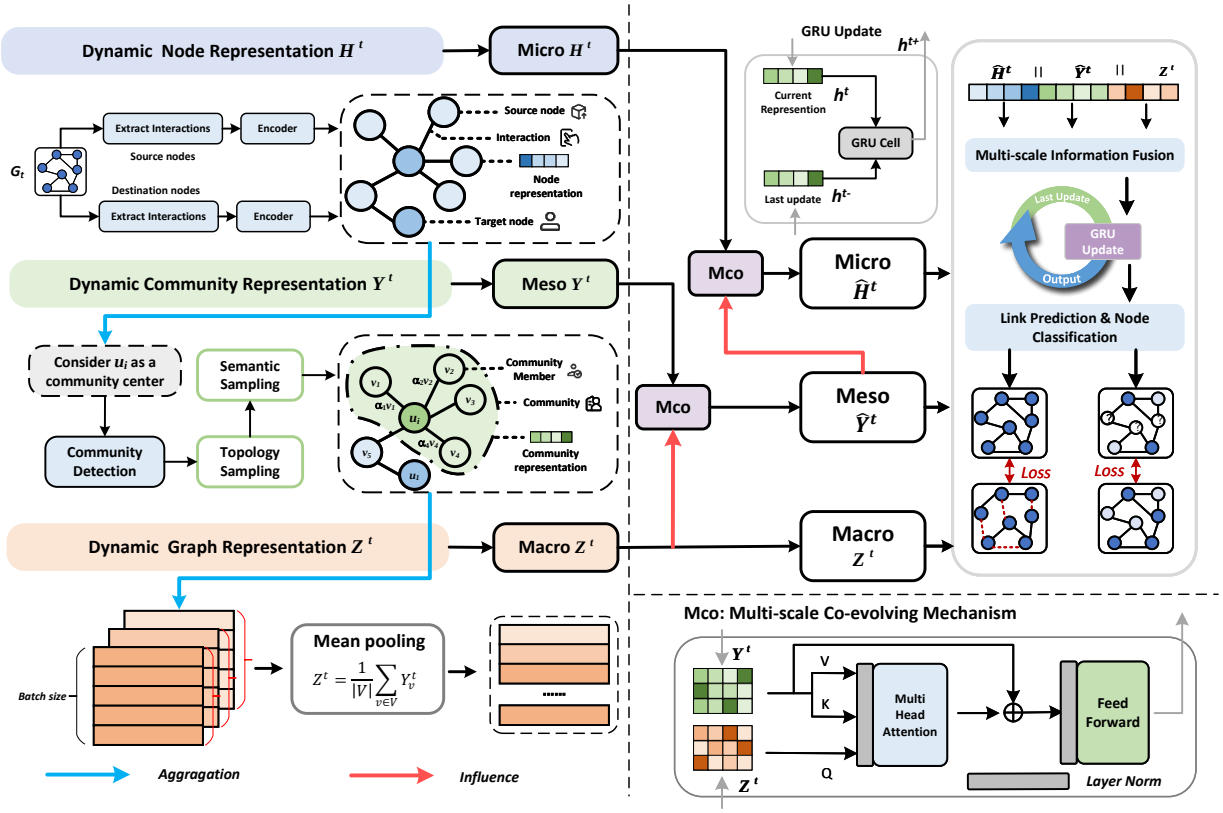


Figure 2: A conceptual diagram of the ComCo architecture. **Left:** Interactions are extracted to generate Microscopic Representations $\mathbf{H}^t$, a community detection mechanism is proposed to obtain Mesoscopic Representations $\mathbf{Y}^t$, which are then averaged to obtain Macroscopic Representations $\mathbf{Z}^t$. **Right:** The Multi-scale Co-evolution Mechanism (Mco) facilitates the influence of macroscopic representations on the learning of mesoscopic representations, which, in turn, guide the learning of microscopic representations. The three structural scales will be fused for downstream tasks.

4.1. Dynamic Node Representation

To capture the long-term changes of nodes and communities, we initialize a node memory vector $\mathbf{s}_N^t \in \mathbb{R}^{n\times d_N}$ and a community memory vector $\mathbf{s}_C^t \in \mathbb{R}^{n\times d_N}$ for each node, where n represents the number of all nodes in the graph. For each interaction behavior (u, v, t), we sample the source node u and the target node v based on temporal proximity to obtain the neighbor sequences S_u^t and S_v^t before time t. For node u, based on the neighbor sequence S_u^t, we extract the features of the neighboring nodes $\mathbf{X}_{u,N}^t \in \mathbb{R}^{|S_u^t|\times d_N}$ and the corresponding edge features $\mathbf{X}_{u,E}^t \in \mathbb{R}^{|S_u^t|\times d_E}$. Following [22], we encode the time interval $\Delta t' = t - t'$ via Bochner's Theorem:

$$\sqrt{\frac{1}{d_T}}[\cos(\omega_1\Delta t'), \sin(\omega_1\Delta t'), \ldots, \cos(\omega_{d_T}\Delta t'), \sin(\omega_{d_T}\Delta t')], \tag{1}$$

where $\omega_1, \ldots, \omega_{d_T}$ are learnable parameters. The time interval encodings of interactions are denoted by $\mathbf{X}_{u,T}^t \in \mathbb{R}^{(|S_u^t|\times d_T)}$.

The node features, edge features, and temporal encodings are projected into the same space and concatenated to form the final node representation:

$$\mathbf{H}_{u,*}^t = \mathbf{X}_{u,*}^t\mathbf{W}_* + \mathbf{b}_*, \tag{2}$$

$$\mathbf{H}_u^t = \mathbf{H}_{u,N}^t \| \mathbf{H}_{u,E}^t \| \mathbf{H}_{u,T}^t, \tag{3}$$

where * can be N, E, or T, $\mathbf{W}_* \in \mathbb{R}^{d_*\times d}$ represents the learnable parameters, $\mathbf{b}_* \in \mathbb{R}^d$, $\mathbf{H}_u^t \in \mathbb{R}^{|S_u^t|\times 3d}$; we obtain the representation of the target node v in the same way, which is denoted as $\mathbf{H}_v^t \in \mathbb{R}^{|S_v^t|\times 3d}$.

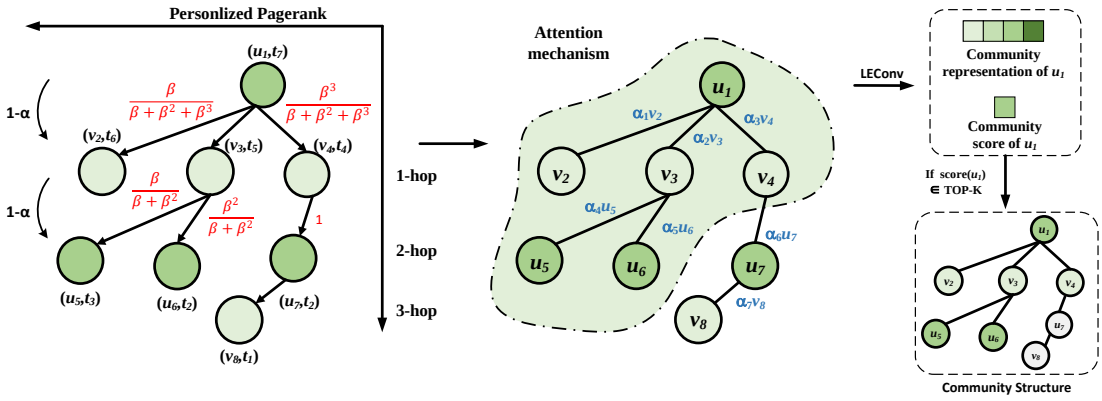


Figure 3: An illustration of the community detection mechanism. The red notes indicate the probabilities of taking different temporal interactions at the next step. The blue notes show the attention scores of the community members.

4.2. Dynamic Community Representation

Acting as a pivotal link between node-level and graph-level representations, community representation contains the most comprehensive information. The goal of our approach is to design a community detection algorithm that captures group preferences, while continuously updating community structures as the network evolves. However, integrating community structure into a deep learning model presents significant challenges, including high computational costs and complex relationships between communities and their corresponding nodes [38]. Despite these challenges, most existing community detection algorithms focus on static graphs, with few attempting to capture the evolving community structures and their temporal characteristics in dynamic graphs.

To bridge the gap in the literature, we propose a novel community detection mechanism for continuous-time dynamic graphs, which takes into account both the topological structure and semantic similarity. We present the process of selecting community members and the detection of communities in Fig. 3.

Specifically, we consider each node involved in the interaction (u, v, t) at any given moment as a potential community center. We maintain a dictionary of topological structure importance, denoted as $\pi_{\text{top}}^{u,t}$, to gauge the closeness of community members to this central node. Furthermore, we evaluate the semantic similarity importance of nodes contained within $\pi_{\text{top}}^{u,t}$ and store this information in a separate dictionary, $\pi_{\text{sem}}^{u,t}$. In this section, taking the source node u as an example, u represents the community center, C_u represents a community, and $N(C_u)$ represents the community members.

Topological structure sampling. At first, we sample multi-order neighbors for each node to identify community members. However, this approach results in an exponential increase in the model's time complexity as the sampling order grows, which can be costly and time-consuming. To address this issue, we introduce Temporal-personalized PageRank [37], which controls the spatial and temporal proximity of community members based on the graph diffusion with parameters α and β, respectively. In light of the decomposition theorem [37], we update and compute the dictionary π_{top}^{u,t^+} using the neighbor nodes of (u, t^+) as keys and the corresponding T-PPR scores as values:

$$\pi_{\text{top}}^{u,t^+} = \frac{m_{u,t^-}\beta}{m_{u,t^-}\beta + \beta}\pi_{\text{top}}^{v,t^-} + \frac{\beta(1-\alpha)}{m_{u,t^-}\beta + \beta}\left(\pi_{\text{top}}^{v,t^-} + \alpha I_{v,t}\right), \tag{4}$$

where α $(0 < \alpha < 1)$ represents the probability of a random walk starting from (u, t) and terminating at (v, τ). β $(0 < \beta < 1)$ is the exponential decay factor that allocates the proportion of transition probabilities to recent temporal neighbors. t^- (t^+) represents the time before (after) t. $m_{u,t^-} = \sum_{z=1}^{|N_{u,t}|} \beta^z$ corresponds to the denominator used in the exponential decay model. $I_{v,t}$ is a dictionary with a single key (v, t) with value 1. The symbol " + " represents the summation of dictionary values that have the same key.

Example 1. *Fig. 3 (left) illustrates the diffusion process of T-PPR on a dynamic graph. Node (u_1, t_7) has three temporal neighbors, and the newly arrived (most recent) neighbor (v_2, t_6) has a T-PPR value calculated with respect to the node*

(u_1, t_7) *as* $\pi_{top}^{1,t_7}(2, t_6) = \frac{\beta}{(\beta^3+\beta^2+\beta)(1-\alpha)\alpha}$. *Moreover, since* (u_8, t_1) *is a sink node, the temporal random walk will terminate at* (u_8, t_1).

Semantic similarity sampling. While topological structure sampling identifies and selects individuals with strong spatio-temporal connections, further filtering is required to identify individuals with semantic similarity. Specifically, for each potential community C_u^t, we construct a representative vector by averaging the representations of the temporal nodes sampled based on $\pi_{\text{top}}^{u,t}$:

$$\mathbf{q}_u^t = \frac{1}{|\pi_{\text{top}}^{u,t}|} \sum_{(v,\tau)\in\pi_{\text{top}}^{u,t}} \mathbf{s}_{N,v}^{\tau}, \tag{5}$$

where $\mathbf{s}_{N,v}^{\tau} \in \mathbb{R}^{3d}$ represents the memory of the community members, $\mathbf{q}_u^t \in \mathbb{R}^{3d}$. Then, we evaluate the semantic similarity of members within their respective communities and explicitly model membership importance using an attention mechanism. These scores are subsequently recorded in the semantic similarity dictionary π_{sem}:

$$\mathbf{Q}_u = \mathbf{q}_u^t \mathbf{W}_q, \mathbf{K}_s = \mathbf{s}_{N,u}^t \mathbf{W}_k, \tag{6}$$

$$\pi_{\text{sem}}^{u,t} = \text{Softmax}\left(\frac{\mathbf{Q}_u \mathbf{K}_s^T}{\sqrt{d}}\right), \tag{7}$$

where $\mathbf{W}_q \in \mathbb{R}^{3d\times 3d}$ and $\mathbf{W}_k \in \mathbb{R}^{3d\times 3d}$ are learnable parameters and $\pi_{\text{sem}}^{u,t}$ represents a dictionary with neighbor nodes as keys and attention coefficients as values.

Dynamic community detection. After constructing the topological structure importance dictionary π_{top} and the semantic similarity importance dictionary π_{sem}, we sum the scores and select the top k community members:

$$\pi^{u,t} = \pi_{\text{sem}}^{u,t} + \pi_{\text{top}}^{u,t}, \tag{8}$$

$$N(C_u^t) = \text{TOPK}\left(\pi^{u,t}, k\right), \tag{9}$$

where the symbol "+" represents the summation of dictionary values with the same key, and the values of $\pi^{u,t}$ represent the strength of community members. We show the Top-k community member sampling strategy in Algorithm 1.

Based on the community member strength, we aggregate the memories of all community members with weights to obtain the final community representation $\mathbf{O}_u^t$, the formula is as follows:

$$w^{u,t}(v,\tau) = \frac{\exp(\pi^{u,t}(v,\tau))}{\sum_{(v,\tau)\in N(C_u^t)} \exp(\pi^{u,t}(v,\tau))}, \tag{10}$$

$$\mathbf{O}_u^t = \sum_{(v,\tau)\in N(C_u^t)} w^{u,t}(v,\tau) \cdot \mathbf{s}_{N,v}^{\tau}, \tag{11}$$

where $\mathbf{s}_{N,v}^{\tau} \in \mathbb{R}^{3d}$ represents the memory of community member v at time τ, $\mathbf{O}_u^t \in \mathbb{R}^{3d}$.

After obtaining possible community representations, we get inspiration from ASAP [39] to select representative community structures and calculate community fitness scores using a fitness function LEConv. This function adaptively captures local extremum information and selects representative clusters across the entire graph. Furthermore, we extend this fitness function to handle continuous-time data, which can be formulated as follows:

$$\varphi_u^t = \sigma(\mathbf{O}_u^t \mathbf{W}_1 + \sum_{v,\tau\in N(C_u^t)} (\mathbf{O}_u^t \mathbf{W}_2 - \mathbf{s}_{N,v}^{\tau} \mathbf{W}_3)), \tag{12}$$

where φ_u^t represents the fitness score of community C_u^t at time t, $\mathbf{O}_u^t \in \mathbb{R}^{3d}$ represents the embedding of community C_u^t.

After calculating the fitness scores for every potential community, function TOPK$(\cdot)$ sorts the fitness scores and gives the indices $I(t)$ of top m selected clusters, finally we store them into the community memory vector:

$$I(t) = \text{TOPK}(\varphi^t, m). \tag{13}$$

$$\mathbf{s}_C^t = \mathbf{O}^t[I(t), :], \tag{14}$$

where $I(t)$ represents the indices of the finally detected communities, and φ^t represents the fitness scores of all potential communities.

To align representations of different structural scales from the same node, we derive the representation of the neighbor sequence from the community memory vector as the final community representation of the node:

$$\mathbf{Y}_u^t = \mathbf{s}_C^t[S_u^t, :], \mathbf{Y}_v^t = \mathbf{s}_C^t[S_v^t, :], \tag{15}$$

where $\mathbf{S}_u^t$ represents the sampled sequence of a node's neighbors. The community representation for node u at time t is given by $\mathbf{Y}_u^t \in \mathbb{R}^{|S_u^t| \times 3d}$, and for node v at time t is given by $\mathbf{Y}_v^t \in \mathbb{R}^{|S_v^t| \times 3d}$.

Algorithm 1: Top-k Community Member Sampling Algorithm

Input: New interaction (u, v, t), node memory s_N^t, weight matrices W_q, W_k, threshold k, community history $\{(\pi^{u,t^-}, m_{u,t^-}) \mid u \in V\}$.

Output: Top-k community member dictionary $\tilde{\pi}^{u,t}$.

1 $\pi_{\text{top}}^{u,t^+} \leftarrow \frac{m_{u,t^-}\beta}{m_{u,t^-}\beta+\beta}\pi_{\text{top}}^{v,t^-} + \frac{\beta(1-\alpha)}{m_{u,t^-}\beta+\beta}\left(\pi_{\text{top}}^{v,t^-} + \alpha I_{v,t}\right)$

2 $q_u^t \leftarrow 0$

3 **for** (v, τ) *in* π_{top}^{u,t^+} **do**

4 $\quad q_u^t \leftarrow q_u^t + s_{N,v}^{\tau}$

5 **end**

6 $q_u^t \leftarrow \frac{q_u^t}{|\pi_{\text{top}}^{u,t^+}|}$

7 $Q_u \leftarrow q_u^t W_q$, $K_s \leftarrow s_{N,u}^t W_k$

8 $\pi_{\text{sem}}^{u,t} \leftarrow \text{Softmax}\left(\frac{Q_u K_s^\top}{\sqrt{d}}\right)$

9 $\pi^{u,t} \leftarrow \pi_{\text{sem}}^{u,t} + \pi_{\text{top}}^{u,t^+}$

10 $\tilde{\pi}^{u,t} \leftarrow$ Select top-k community members from $\pi^{u,t}$

11 **Return** $\tilde{\pi}^{u,t}$

4.3. Complexity Analysis

The cost of merging materialized top-k T-PPR dictionaries for a new interaction (line 1) is $O(k)$. Iterating over the top-k community members incurs an additional $O(k)$. In line 7, the time complexity of the matrix multiplications, including the computation of Q_u, K_s, and V_s, is $O(kd)$. Specifically, if the input dimension is d and the weight matrix has a size of $d \times d$, then multiplying it by the top-k community members results in a complexity of $O(kd)$. The attention mechanism introduces an additional $O(kd)$, while sorting the results incurs a complexity of $O(k \log k)$. Thus, the overall time complexity of the Top-k Community Member Sampling Algorithm is $O(kd + k \log k)$.

4.4. Dynamic Graph Representation

Following the previous works, we apply average pooling to the community representations to derive a graph-level representation. The formula is as follows:

$$\mathbf{Z}_u^t = \frac{1}{m}\sum_{u \in I(t)} \mathbf{Y}_u^t, \mathbf{Z}_v^t = \frac{1}{m}\sum_{v \in I(t)} \mathbf{Y}_v^t. \tag{16}$$

The aggregation process from node representation to graph representation is illustrated in Fig. 4. The effectiveness of this approach lies in that high-level predictions offer a macroscopic perspective [40], which enables the correction of potential biases present in low-level (or small-scale) predictions. From an information theory perspective, at higher structural levels, information becomes more condensed and less noisy, making the temporal dynamics of these scales more predictable compared to lower scales [13].

Nevertheless, it is only a foundational step in modeling the aggregation relationship from lower-scale to higher-scale representations. The primary focus of this paper is to learn the intrinsic correlations across structural scales. To achieve this, we propose a Multi-scale Co-Evolution module.

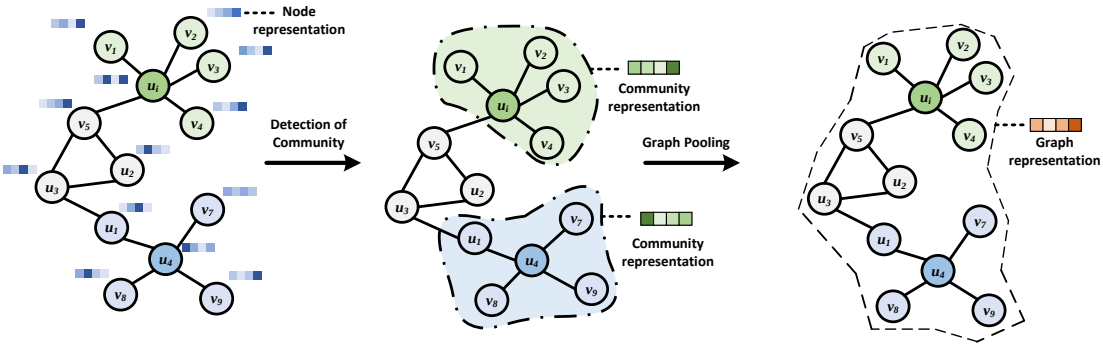


Figure 4: From left to right: the aggregation process of microscopic, mesoscopic, and macroscopic representation simulations.

4.5. Multi-scale Co-Evolution for Fusion3M

After obtaining the node representations $\mathbf{H}_*^t$, community representations $\mathbf{Y}_*^t$, and graph representations $\mathbf{Z}_*^t$ (with $*$ indicating either the source node u or the target node v), we design a multi-scale co-evolution module to learn the consistency between different scales. To accomplish this objective, we employ a Transformer-based encoder [41], which enables higher-level representations to guide the learning process of lower-level representations. Finally, these representations are concatenated and fused with a recurrent neural network for integration and updating.

Transformer Encoder. Following the work in [9], we concatenate the representations of source node u and target node v at each scale, allowing the node representations to learn the temporal dependencies within and across the neighbors of u and v. Next, the Transformer encoder guides the evolution of the community representation by the graph representation, and the node representation by the community representation. Specifically, as shown in Fig. 2, where $\mathbf{Q}$ denotes the 'queries', $\mathbf{K}$ denotes the 'keys', and $\mathbf{V}$ denotes the 'values'. We feed the higher-level components as the query $\mathbf{Q}$ and utilize the lower-level components as the key $\mathbf{K}$ and the value $\mathbf{V}$ in the multi-head attention framework.

Taking the evolution of node representation as an example, for node u, we use the community representation $\mathbf{Y}^t$ as the $\mathbf{Q}$, and the node representation $\mathbf{H}^t$ as the $\mathbf{K}$ and $\mathbf{V}$. The formula is organized as follows:

$$\begin{gathered}\tilde{\mathbf{H}}^t = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V}, \\ \mathbf{Q} = \text{LN}(\tilde{\mathbf{Y}}^t\mathbf{W}_Q), \quad \mathbf{K} = \text{LN}(\mathbf{H}^t\mathbf{W}_K), \\ \mathbf{V} = \text{LN}(\mathbf{H}^t\mathbf{W}_V),\end{gathered} \tag{17}$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{3d\times 3d}$ are the learnable parameters used to project the 'Q-K-V', and LN$(\cdot)$ represents layer normalization. $\tilde{\mathbf{Y}}^t$ represents the community representation evolved in the same manner. Note that $\mathbf{H}^t$ and $\hat{\mathbf{Y}}^t$ both have dimensions in $\mathbb{R}^{(|S_u^t|+|S_v^t|)\times 3d}$, indicating that each level of representation encompasses embeddings of $|S_u^t|+|S_v^t|$ neighboring sequences. With the dot-product attention mechanism, ComCo can determine how to enhance the node representation by assimilating community embeddings from the node's temporal neighbors, thereby capturing the underlying correlations between the node and community representations.

Subsequently, we feed the node representation $\tilde{\mathbf{H}}^t$ into a Feed-Forward Network (FFN), the formula of which is organized as follows:

$$\text{FFN}(\mathbf{H}) = \text{GELU}(\mathbf{H}\mathbf{W}_{FFN_1} + \mathbf{b}_{FFN_1})\mathbf{W}_{FFN_2} + \mathbf{b}_{FFN_2}, \tag{18}$$

$$\hat{\tilde{\mathbf{H}}}^t = \text{FFN}(\text{LN}(\tilde{\mathbf{H}}^t) + \mathbf{H}^t) + \tilde{\mathbf{H}}^t, \tag{19}$$

where $\mathbf{W}_{FFN_1}, \mathbf{W}_{FFN_2} \in \mathbb{R}^{3d\times 3d}$, $\mathbf{b}_{FFN_1}, \mathbf{b}_{FFN_2} \in \mathbb{R}^{3d}$ are learnable parameters, and $\hat{\tilde{\mathbf{H}}}^t \in \mathbb{R}^{(|S_u^t|+|S_v^t|)\times 3d}$. LN$(\cdot)$ represents layer normalization. We obtain the final community representation $\hat{\tilde{\mathbf{Y}}}^t$ and graph representation $\tilde{\mathbf{Z}}^t$ in the same manner.

Once we have obtained representations of different structural scales for each node, we integrate these representations to facilitate downstream tasks. This integration is achieved by concatenating the three representations and then applying an averaging operation:

$$\begin{aligned} \mathbf{h}^t &= \mathrm{MEAN}([\hat{\tilde{\mathbf{H}}}^t \| \hat{\tilde{\mathbf{Y}}}^t \| \tilde{\mathbf{Z}}^t])\mathbf{W}_{\mathrm{out}} + \mathbf{b}_{\mathrm{out}}, \\ \mathbf{h}_u^t &= \mathbf{h}^t[:\ l_u^t, :], \\ \mathbf{h}_v^t &= \mathbf{h}^t[l_u^t\ :\ l_u^t + l_v^t, :], \end{aligned} \tag{20}$$

Memory Fusion and Updating. Memory serves to record the interaction history between nodes. Previous research treats each batch of data as an isolated entity during training, overlooking the evolving patterns for the same node across various batches. The memory update module addresses this oversight, specifically, we fuse the multi-scale node representation $\mathbf{h}_u^t$ and the memory vector $\mathbf{s}_{N,u}^t$ by GRU as follows:

$$\mathbf{s}_{N,u}^t = \mathrm{GRU}(\mathbf{h}_u^t, \mathbf{s}_{N,u}^{t-}), \tag{21}$$

the target node v can also be updated in the same manner.

4.6. Prediction and Objective Function

We adopt a 2-layer multilayer perceptron (MLP) and use softmax to convert node representations into probability. To predict whether there will be a link between two nodes u and v at time t, the two temporal embeddings should be concatenated as $(\mathbf{h}_u^t \| \mathbf{h}_v^t)$:

$$\hat{y} = \mathrm{Softmax}\left(\mathrm{MLP}\left(\mathrm{ReLU}\left(\mathrm{MLP}(\mathbf{h}_u^t \| \mathbf{h}_v^t)\right)\right)\right). \tag{22}$$

To predict the dynamic node state, we only need to input the node representation $\mathbf{h}_*^t$ into the MLP. The formulation is as follows:

$$\hat{y} = \mathrm{Softmax}\left(\mathrm{MLP}\left(\mathrm{ReLU}\left(\mathrm{MLP}(\mathbf{h}_*^t)\right)\right)\right), \tag{23}$$

where $*$ could be u or v. We use the binary cross-entropy function as the loss function for link prediction or node classification, which is defined as:

$$\mathcal{L} = -\frac{1}{K}\sum_{i=1}^{K}\left(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right), \tag{24}$$

where K denotes the number of samples, y_i represents the ground-truth label of i-th sample and $\hat{y}_i$ represents the prediction value.

5. Experiments

In this section, we present a series of experiments designed to evaluate the ComCo approach across seven real-world datasets. These experiments aim to address the following five research questions (RQs):

- **RQ1**: How does ComCo perform when compared with state-of-the-art methods? (Section 5.2)
- **RQ2**: How do various components of ComCo contribute to its performance? (Section 5.3)
- **RQ3**: What can be reflected from the evolution of community representations? (Section 5.4)
- **RQ4**: How does the generality of the memory update module in other approaches? (Section 5.5)
- **RQ5**: How do the selections of key hyperparameters influence the performance of ComCo? (Section 5.6)

5.1. Experimental Settings

Evaluation Tasks and Metrics. We follow [42] to evaluate the link prediction model with transductive and inductive setups. The transductive setup predicts future links between nodes that are observed during the training period, while the inductive setup predicts future links between nodes that have not appeared during the training period; the latter is more challenging. We use Mean Average Precision (AP) and Receiver Operating Characteristic Curve (ROC-AUC) as evaluation metrics. For each metric, we adopt the random (rnd), historical (hist), and inductive (ind) negative sampling strategies as in [42] for evaluation. We also follow [22] for dynamic node classification, which estimates the state of a node in a given interaction at a specific time. Due to label imbalance, we use ROC-AUC as the evaluation metric.

Training Configurations. For both tasks, we optimize all models using Adam and employ binary cross-entropy loss as the objective function. We train the models for 50 epochs and utilize an early stopping strategy with a patience of 10. We select the model that achieves the best performance on the validation set for testing. For all methods on all datasets, we set the batch size to 200. We run the experiments five times with different random seeds and provide the average performance to eliminate bias. We split each dataset chronologically with a training/validation/testing ratio of 70%/15%/15%.

Parameter Implementations. For all datasets, we set the time encoding dimension d_T to 100 and the aligned encoding dimension d to 50. The termination probability α is set to 0.4, and the exponential decay factor β is 0.8. The maximum number of community members k is 32. We sample 32 neighbors using the recent strategy. The dropout rate is 0.1, and the number of attention heads is 2.

Baselines and Datasets. We perform experiments across seven datasets spanning diverse domains, including six publicly accessible datasets[1] and a proprietary Blockchain dataset sourced from Xblock[2]. These datasets encompass a wide range of real-world applications, providing valuable insights for our study. Notably, none of these datasets include node attributes. Comprehensive details regarding these datasets are presented in Table 2.

- **Wikipedia**: The Wikipedia edit tracking dataset is a bipartite graph that captures user interactions with Wikipedia pages and includes dynamic labels indicating whether users are temporarily banned from editing.
- **Reddit**: The Reddit dataset forms a bipartite graph of user posting activities within subreddits, complete with LIWC features and dynamic user ban information.
- **MOOC**: The MOOC dataset is a bipartite network illustrating student engagement with course content, featuring a compact set of features for each interaction.
- **Enron**: The Enron dataset logs email communications between employees, providing a raw view of workplace interactions without additional attributes.
- **Social Evo.**: The Social Evo. dataset monitors the daily activities of a dormitory through a mobile proximity network, capturing social interactions with simple feature sets.
- **UCI**: The UCI dataset represents an online communication network among students, with timestamped messages forming links between nodes.
- **Blockchain (New)**: This dataset comprises accounts identified as phishing within the Ethereum network, along with a subset of unmarked accounts, serving as central nodes. By leveraging the Etherscan API, we capture their first-order neighbors and the transactions that occurred between these nodes. Considering the expansive nature of the entire graph, our analysis was confined to the first-order transaction network.

We compare ComCo with nine popular continuous-time dynamic graph learning baselines that are based on recurrent neural networks, random walks, and self-attention models, including JODIE, DyRep, TGAT, TGN, CAWN, GraphMixer, DygFormer, FreeDyg and CorDGT.

- **JODIE (2019) [23]** utilizes recurrent neural networks for updating embeddings and introduces a novel projection operator to predict future source-target interactions.

[1] https://zenodo.org/records/7213796#.Y1c06y8r30o
[2] https://xblock.pro/

Table 2
Dataset statistics.

Datasets	Domain	Nodes	Edges	Time Granularity	Bipartite	Unique Steps	Duration
Wikipedia	Social	9,227	157,474	Unique timestamp	True	152,757	1 month
Reddit	Social	10,984	672,447	Unique timestamp	True	669,065	1 month
Mooc	Interaction	7,144	411,749	Unique timestamp	True	345,600	17 month
Enron	Interaction	184	125,235	Unique timestamp	False	22,632	3 years
Social Evo.	Social	74	2,099,519	Unique timestamp	False	565,932	8 month
UCI	Proximity	1,899	59,835	Unique timestamp	False	58,911	196 days
Blockchain	Financial	16,884	95,328	Unique timestamp	False	672,708	136 days

Table 3
AP for transductive dynamic link prediction with different sampling strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	GraphMixer	DygFormer	FreeDyg	CorDGT	Ours
random	Wikipedia	96.60 ± 0.21	93.23 ± 0.31	94.85 ± 0.11	98.35 ± 0.02	97.75 ± 0.36	96.84 ± 0.05	98.49 ± 0.03	98.47 ± 0.05	98.16 ± 0.11	**99.83 ± 0.24**
	Reddit	98.34 ± 0.10	97.14 ± 0.24	96.61 ± 0.11	98.68 ± 0.24	99.43 ± 0.16	97.37 ± 0.01	98.67 ± 1.43	99.43 ± 0.82	99.35 ± 0.26	**99.56 ± 0.43**
	Mooc	82.11 ± 0.87	81.04 ± 0.31	80.69 ± 0.22	**91.59 ± 0.12**	81.53 ± 0.22	82.49 ± 0.03	85.43 ± 2.27	90.63 ± 0.15	73.12 ± 0.15	81.85 ± 0.43
	Enron	84.39 ± 1.97	83.42 ± 0.19	70.09 ± 0.22	87.72 ± 0.37	90.35 ± 0.24	82.03 ± 0.07	91.34 ± 0.36	91.34 ± 0.52	86.63 ± 0.36	**94.06 ± 0.54**
	Social Evo.	82.41 ± 1.07	88.43 ± 0.24	90.03 ± 0.19	93.46 ± 0.52	84.43 ± 0.14	91.61 ± 0.19	92.35 ± 0.36	88.29 ± 0.17	93.57 ± 0.21	**96.90 ± 0.46**
	UCI	87.38 ± 1.95	63.38 ± 0.16	79.03 ± 1.04	93.13 ± 0.24	94.42 ± 1.04	93.36 ± 0.10	95.81 ± 0.33	95.25 ± 0.10	94.79 ± 0.25	**98.95 ± 0.12**
	Blockchain	97.51 ± 0.26	96.09 ± 0.64	86.84 ± 0.86	98.46 ± 0.09	99.23 ± 0.01	97.35 ± 0.03	98.91 ± 0.23	88.96 ± 0.17	97.07 ± 0.15	**99.37 ± 0.02**
average rank		7.14	8.57	9.00	4.14	5.07	6.43	3.21	4.29	5.43	**1.71**
historical	Wikipedia	82.91 ± 1.29	78.63 ± 0.55	87.70 ± 0.85	87.22 ± 0.33	72.33 ± 0.35	90.96 ± 0.32	75.38 ± 2.48	81.73 ± 0.54	**98.44 ± 0.17**	88.78 ± 0.43
	Reddit	80.53 ± 0.29	78.46 ± 0.52	76.28 ± 0.12	80.24 ± 0.24	81.54 ± 0.52	78.03 ± 0.73	78.65 ± 0.55	**82.25 ± 0.92**	81.25 ± 0.42	80.78 ± 0.54
	Mooc	76.05 ± 4.08	74.26 ± 0.15	76.93 ± 0.68	86.06 ± 0.17	75.53 ± 0.25	75.94 ± 1.07	81.45 ± 0.34	82.32 ± 0.41	82.53 ± 0.11	**92.76 ± 0.05**
	Enron	71.63 ± 6.47	70.38 ± 0.15	62.62 ± 0.39	74.27 ± 1.48	62.42 ± 0.24	79.15 ± 0.05	80.26 ± 0.43	72.26 ± 0.15	**86.27 ± 0.11**	80.02 ± 0.14
	Social Evo.	77.57 ± 9.90	92.63 ± 0.24	89.89 ± 0.40	93.53 ± 0.64	86.45 ± 0.83	92.80 ± 0.20	94.46 ± 1.34	92.74 ± 0.71	92.41 ± 0.72	**95.18 ± 0.03**
	UCI	74.40 ± 3.39	56.23 ± 0.94	67.56 ± 0.99	79.21 ± 0.18	66.36 ± 0.53	84.36 ± 0.17	81.06 ± 0.46	82.17 ± 0.37	86.11 ± 0.24	**88.53 ± 0.76**
	Blockchain	88.04 ± 0.94	88.29 ± 0.13	78.18 ± 1.26	84.30 ± 0.72	80.26 ± 2.53	93.67 ± 0.71	80.16 ± 0.63	81.25 ± 0.81	90.97 ± 0.21	**93.42 ± 0.74**
average rank		6.71	7.71	7.86	4.71	8.14	4.43	5.57	4.86	2.86	**2.14**
inductive	Wikipedia	75.12 ± 0.17	72.46 ± 0.35	86.93 ± 1.55	86.87 ± 0.01	73.53 ± 0.52	87.87 ± 0.15	75.13 ± 0.43	83.62 ± 0.59	**97.31 ± 0.20**	81.66 ± 0.14
	Reddit	86.85 ± 0.52	85.36 ± 0.31	84.59 ± 0.10	89.95 ± 0.16	90.62 ± 0.53	85.66 ± 0.32	85.43 ± 1.44	86.03 ± 0.19	83.85 ± 0.52	**99.60 ± 0.54**
	Mooc	63.60 ± 0.47	60.59 ± 0.72	73.64 ± 1.10	77.96 ± 0.18	74.43 ± 0.42	72.85 ± 0.41	79.45 ± 0.54	79.93 ± 0.27	76.73 ± 0.13	**84.98 ± 0.17**
	Enron	70.47 ± 2.62	67.67 ± 0.53	63.34 ± 0.21	73.44 ± 0.46	76.53 ± 1.04	75.61 ± 0.26	76.41 ± 0.36	72.58 ± 0.74	**85.95 ± 0.13**	80.69 ± 1.19
	Social Evo.	79.40 ± 6.30	93.45 ± 0.24	89.95 ± 0.40	94.35 ± 0.24	87.53 ± 0.14	92.76 ± 0.13	95.63 ± 0.32	90.16 ± 0.26	93.16 ± 0.15	**95.90 ± 0.04**
	UCI	65.85 ± 1.80	53.35 ± 0.25	66.97 ± 0.55	70.74 ± 0.68	65.42 ± 0.54	**79.39 ± 0.20**	72.25 ± 0.13	68.63 ± 0.27	70.56 ± 0.21	70.73 ± 0.58
	Blockchain	86.01 ± 1.04	90.40 ± 0.43	86.16 ± 0.24	83.12 ± 1.04	81.51 ± 3.17	88.39 ± 0.23	82.09 ± 0.42	75.52 ± 0.33	88.03 ± 0.17	**92.35 ± 0.14**
average rank		7.57	7.57	7.00	4.29	6.71	4.43	4.71	6.00	4.43	**2.29**

- **DyRep (2019) [24]** proposes a bi-temporal deep temporal point process model with a temporal attention network to encode the dynamics of graph evolution.
- **TGAT (2020)[22]** encodes temporal features using Bochner's theorem and aggregates neighbor features with self-attention for each node.
- **TGN (2020) [7]** maintains an evolving memory for each node, updating it through a recurrent neural network with historical and current interaction data.
- **CAWN (2021) [11]** is a hybrid RNN and self-attention model that represents network dynamics by aggregating temporal network motifs and encoding them for downstream tasks.
- **GraphMixer (2023) [8]** simplifies graph processing with an MLP-based model and introduces a non-trainable time encoding for explicit temporal discrimination.
- **DygFormer (2023) [9]** enhances link prediction by encoding common neighbors and using a transformer encoder to aggregate node representations across source and target node neighbors.
- **FreeDyg (2024) [43]** introduces a frequency-enhanced approach to dynamic graph learning by transforming node interactions into time series analysis.

Table 4
ROC-AUC for transductive dynamic link prediction with different sampling strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	GraphMixer	DygFormer	FreeDyg	CorDGT	Ours
random	Wikipedia	96.38 ± 0.19	93.14 ± 1.43	93.95 ± 0.10	98.29 ± 0.01	98.41 ± 0.42	96.49 ± 0.04	98.91 ± 1.09	98.62 ± 0.21	99.36 ± 0.13	**99.79 ± 0.14**
	Reddit	98.29 ± 0.08	97.14 ± 0.57	96.40 ± 0.11	98.75 ± 0.04	98.94 ± 0.54	97.27 ± 0.01	98.31 ± 0.03	98.59 ± 0.12	**99.42 ± 0.14**	98.15 ± 0.15
	Mooc	85.06 ± 1.11	86.03 ± 0.41	81.64 ± 0.18	**93.16 ± 0.12**	81.16 ± 1.04	83.82 ± 0.10	85.02 ± 0.31	87.23 ± 0.39	72.12 ± 0.22	86.04 ± 0.04
	Enron	86.51 ± 1.80	83.41 ± 1.54	66.15 ± 0.40	89.55 ± 0.30	89.03 ± 0.41	84.28 ± 0.04	91.31 ± 0.57	91.36 ± 0.61	86.04 ± 0.23	**92.36 ± 0.05**
	Social Evo.	85.63 ± 0.93	91.14 ± 0.43	91.49 ± 0.13	94.35 ± 0.43	86.19 ± 0.14	93.75 ± 0.08	95.31 ± 0.45	92.44 ± 0.72	92.45 ± 0.31	**97.89 ± 0.08**
	UCI	89.29 ± 0.79	70.42 ± 0.15	79.68 ± 1.28	92.92 ± 0.31	91.87 ± 0.41	91.94 ± 0.14	94.57 ± 0.04	95.94 ± 0.16	94.16 ± 0.17	**98.69 ± 0.20**
	Blockchain	97.60 ± 0.20	96.09 ± 0.88	87.21 ± 0.68	98.55 ± 0.07	99.17 ± 0.02	97.21 ± 0.02	98.56 ± 0.64	92.79 ± 0.18	98.14 ± 0.24	**99.29 ± 0.02**
average rank		7.00	8.29	9.00	3.71	5.57	6.71	3.57	4.14	4.86	**2.14**
historical	Wikipedia	80.93 ± 0.84	76.12 ± 0.14	82.80 ± 1.31	82.94 ± 0.41	68.14 ± 0.54	87.80 ± 0.46	74.20 ± 3.46	81.41 ± 0.17	**98.26 ± 0.10**	88.39 ± 0.04
	Reddit	80.93 ± 0.29	81.19 ± 1.64	76.00 ± 0.03	80.05 ± 0.14	79.04 ± 0.16	77.34 ± 0.45	87.54 ± 0.32	82.19 ± 0.36	82.26 ± 0.36	**87.02 ± 0.05**
	Mooc	79.89 ± 1.10	81.04 ± 0.65	71.31 ± 0.58	**87.46 ± 2.52**	70.43 ± 0.52	74.55 ± 2.09	85.04 ± 0.43	87.23 ± 0.27	81.22 ± 0.19	86.03 ± 0.01
	Enron	75.51 ± 3.38	75.93 ± 0.73	58.53 ± 0.69	78.49 ± 0.43	77.14 ± 1.52	76.68 ± 0.56	80.52 ± 0.13	75.83 ± 0.83	**86.85 ± 0.19**	81.04 ± 0.02
	Social Evo.	81.57 ± 5.33	92.41 ± 0.14	85.87 ± 0.36	93.64 ± 0.24	92.19 ± 1.04	90.95 ± 0.17	96.89 ± 0.95	76.34 ± 0.92	92.94 ± 0.21	**99.01 ± 0.07**
	UCI	79.47 ± 2.18	60.43 ± 0.75	57.40 ± 1.58	76.06 ± 0.48	75.27 ± 1.04	78.06 ± 0.10	76.97 ± 0.14	78.36 ± 0.17	**86.31 ± 0.15**	83.57 ± 0.32
	Blockchain	88.96 ± 1.12	88.47 ± 0.14	80.93 ± 1.27	84.07 ± 0.10	74.33 ± 1.83	94.26 ± 0.42	76.02 ± 0.36	83.58 ± 0.52	90.1 ± 0.18	**94.62 ± 0.29**
average rank		6.43	6.43	8.57	4.57	8.14	5.71	4.86	5.86	2.57	**1.86**
inductive	Wikipedia	70.46 ± 0.36	66.38 ± 1.04	81.79 ± 2.02	82.09 ± 0.17	79.94 ± 0.41	83.14 ± 0.22	71.78 ± 0.52	75.52 ± 0.31	**97.06 ± 0.17**	80.03 ± 0.42
	Reddit	83.41 ± 0.39	82.04 ± 0.95	81.85 ± 0.04	**86.06 ± 0.24**	82.54 ± 1.04	82.96 ± 0.32	82.94 ± 0.31	82.15 ± 0.52	82.15 ± 0.41	83.04 ± 0.43
	Mooc	64.00 ± 0.61	63.24 ± 0.62	68.51 ± 0.43	78.37 ± 0.11	76.36 ± 2.25	71.34 ± 0.30	79.49 ± 0.33	77.17 ± 0.25	74.67 ± 0.31	**81.43 ± 0.04**
	Enron	71.13 ± 1.28	65.29 ± 0.41	58.54 ± 0.51	73.74 ± 0.52	72.43 ± 1.42	72.37 ± 0.16	74.42 ± 0.19	73.54 ± 032	**85.62 ± 0.31**	76.41 ± 0.94
	Social Evo.	81.75 ± 3.83	92.01 ± 0.15	85.82 ± 0.38	92.75 ± 0.42	96.15 ± 0.96	90.72 ± 0.03	95.53 ± 0.32	86.92 ± 0.27	93.62 ± 0.19	**98.92 ± 0.04**
	UCI	64.87 ± 1.45	52.41 ± 1.04	57.61 ± 1.05	64.44 ± 1.04	63.04 ± 1.65	**74.45 ± 0.03**	65.41 ± 0.10	71.34 ± 0.21	70.96 ± 0.62	68.34 ± 0.71
	Blockchain	85.26 ± 1.05	91.01 ± 0.23	87.97 ± 0.17	79.61 ± 2.74	75.54 ± 2.44	88.34 ± 0.48	77.81 ± 0.04	76.53 ± 0.25	88.39 ± 0.12	**93.54 ± 0.25**
average rank		7.14	8.00	7.86	4.29	6.14	4.57	4.86	6.07	3.64	**2.43**

Table 5
AP for inductive dynamic link prediction with different sampling strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	GraphMixer	DygFormer	FreeDyg	CorDGT	Ours
random	Wikipedia	94.83 ± 0.09	91.34 ± 0.14	94.54 ± 0.01	97.64 ± 0.01	97.91 ± 0.19	96.31 ± 0.04	98.69 ± 0.32	98.68 ± 0.29	98.13 ± 0.22	**99.33 ± 0.32**
	Reddit	96.68 ± 0.06	93.14 ± 1.34	93.92 ± 0.30	96.24 ± 0,24	97.14 ± 0.06	95.14 ± 0.03	97.31 ± 0.19	**98.63 ± 0.57**	98.52 ± 0.33	96.64 ± 0.64
	Mooc	80.94 ± 1.06	82.04 ± 1.91	78.79 ± 0.31	**90.45 ± 1.07**	82.14 ± 0.42	80.98 ± 0.10	84.64 ± 0.94	87.52 ± 0.07	72.83 ± 0.09	85.54 ± 0.47
	Enron	78.88 ± 1.32	72.54 ± 0.15	65.19 ± 0.53	75.67 ± 0.06	86.14 ± 0.09	75.09 ± 0.16	88.32 ± 0.52	83.32 ± 0.76	88.62 ± 0.14	**93.61 ± 0.65**
	Social Evo.	83.57 ± 0.43	92.14 ± 0.32	87.23 ± 0.25	89.75 ± 0.32	79.38 ± 0.53	89.16 ± 0.03	92.14 ± 0.31	86.23 ± 0.22	94.14 ± 0.41	**95.85 ± 0.08**
	UCI	77.13 ± 2.10	53.41 ± 1.61	79.71 ± 0.87	89.20 ± 0.18	93.14 ± 0.14	91.42 ± 0.08	94.34 ± 0.39	92.23 ± 0.25	90.29 ± 0.23	**97.02 ± 0.05**
	Blockchain	93.31 ± 0.05	92.22 ± 0.34	84.06 ± 0.38	95.87 ± 0.11	97.11 ± 0.02	90.41 ± 0.14	96.64 ± 0.46	76.93 ± 0.14	94.90 ± 0.18	**99.17 ± 0.31**
average rank		7.29	7.93	8.71	5.29	4.71	7.00	2.93	4.71	4.43	**2.00**
historical	Wikipedia	68.47 ± 0.39	65.29 ± 0.41	86.15 ± 0.48	83.14 ± 0.66	68.13 ± 0.31	87.93 ± 0.39	67.48 ± 2.97	74.41 ± 0.05	**90.38 ± 0.32**	73.45 ± 0.30
	Reddit	62.51 ± 0.50	59.19 ± 1.81	62.94 ± 0.21	64.14 ± 0.14	62.14 ± 0.41	64.50 ± 0.78	61.34 ± 0.43	63.97 ± 0.87	**81.42 ± 0.25**	63.36 ± 0.62
	Mooc	62.10 ± 0.72	63.04 ± 1.03	73.50 ± 0.77	76.02 ± 0.09	77.17 ± 0.25	72.43 ± 0.15	**77.64 ± 0.19**	74.53 ± 0.17	72.81 ± 0.31	75.63 ± 0.36
	Enron	66.58 ± 1.65	64.03 ± 0.63	59.04 ± 0.66	60.82 ± 1.67	61.75 ± 0.17	74.32 ± 0.30	73.91 ± 0.14	65.99 ± 0.11	79.06 ± 0.31	**80.52 ± 0.52**
	Social Evo.	80.37 ± 2.93	86.91 ± 0.14	88.38 ± 0.87	90.57 ± 0.32	79.01 ± 0.11	91.82 ± 0.71	93.42 ± 1.42	89.61 ± 0.55	91.95 ± 0.19	**95.67 ± 0.04**
	UCI	63.84 ± 2.57	55.93 ± 1.42	69.05 ± 0.45	71.38 ± 0.01	65.01 ± 1.19	**81.30 ± 0.18**	71.19 ± 0.44	68.94 ± 0.23	68.86 ± 0.14	69.53 ± 0.73
	Blockchain	75.50 ± 1.07	81.22 ± 0.42	78.86 ± 0.53	76.32 ± 3.27	74.86 ± 2.45	83.91 ± 0.00	75.37 ± 0.36	70.08 ± 0.12	84.65 ± 0.24	**88.31 ± 0.63**
average rank		7.71	8.29	6.00	4.57	7.57	3.29	5.14	6.00	3.29	**3.14**
inductive	Wikipedia	68.46 ± 0.40	62.14 ± 0.42	86.16 ± 0.43	83.14 ± 0.67	68.42 ± 0.41	87.93 ± 0.39	67.91 ± 0.14	74.41 ± 0.35	**88.26 ± 0.29**	73.74 ± 0.74
	Reddit	62.50 ± 0.50	62.02 ± 1.91	62.94 ± 0.21	63.03 ± 0.24	62.14 ± 1.04	64.49 ± 0.78	63.03 ± 0.33	63.97 ± 0.74	**84.36 ± 0.51**	66.74 ± 0.73
	Mooc	62.11 ± 0.72	63.91 ± 0.13	73.33 ± 1.00	76.02 ± 0.10	74.19 ± 0.94	72.43 ± 0.15	76.28 ± 1.84	72.91 ± 0.52	73.94 ± 0.19	**77.64 ± 1.64**
	Enron	66.58 ± 1.65	62.79 ± 0.14	59.06 ± 0.68	60.82 ± 1.67	61.35 ± 0.41	74.33 ± 0.28	73.43 ± 0.14	65.08 ± 0.16	78.26 ± 0.11	**80.90 ± 1.31**
	Social Evo.	80.37 ± 2.93	87.91 ± 0.31	88.38 ± 0.87	90.56 ± 0.65	79.14 ± 0.57	91.81 ± 0.71	94.19 ± 0.71	89.61 ± 0.21	93.14 ± 0.12	**95.73 ± 0.28**
	UCI	63.84 ± 2.56	52.14 ± 1.41	69.06 ± 0.47	71.04 ± 0,01	64.54 ± 0.12	**81.33 ± 0.18**	71.31 ± 1.43	68.91 ± 1.23	70.85 ± 0.31	71.54 ± 0.35
	Blockchain	75.50 ± 1.06	81.11 ± 0.65	78.86 ± 0.42	76.30 ± 3.27	74.87 ± 2.45	83.90 ± 0.02	75.09 ± 0.02	70.06 ± 0.82	84.95 ± 0.52	**87.64 ± 0.36**
average rank		7.86	8.29	6.29	5.21	8.00	3.43	4.79	6.43	2.71	**2.00**

- **CorDGT (2025) [44]** introduces a novel positional encoding with parameter-free interaction intensity estimation, enabling efficient preservation of evolving high-order proximity for node representation learning.

5.2. Performance Comparison (RQ1)

We present the performance of various methods on the AP metric for transductive and inductive dynamic link prediction in Table 3 and Table 5, respectively. Similarly, the performance on the ROC-AUC metric is summarized in Table 4 and Table 6. The evaluation includes three negative sampling strategies. The best and suboptimal results are highlighted in **bold** and underlined fonts, respectively. The results are multiplied by 100 for better display layout.

During the evaluation phase, we combine the original test set (considered as positive samples) with additional negative samples. We employ three different negative sampling strategies, namely: 1) Random Negative Sampling

Table 6
ROC-AUC for inductive dynamic link prediction with different sampling strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	GraphMixer	DygFormer	FreeDyg	CorDGT	Ours
random	Wikipedia	94.32 ± 0.05	90.15 ± 0.15	93.68 ± 0.03	97.59 ± 0.04	97.04 ± 0.43	95.94 ± 0.03	99.48 ± 0.84	97.22 ± 0.41	97.82 ± 0.14	**99.69 ± 0.47**
	Reddit	96.62 ± 0.04	94.06 ± 0.65	93.58 ± 0.36	96.58 ± 0.02	**98.49 ± 0.03**	94.92 ± 0.02	96.48 ± 0.52	97.39 ± 0.53	98.14 ± 0.15	97.74 ± 0.84
	Mooc	83.57 ± 1.45	82.62 ± 1.32	79.81 ± 0.33	**91.91 ± 1.49**	82.83 ± 0.15	82.48 ± 0.21	86.53 ± 0.25	87.35 ± 0.11	74.78 ± 0.12	87.47 ± 0.25
	Enron	79.47 ± 0.98	75.27 ± 0.48	61.01 ± 0.85	77.72 ± 0.09	86.49 ± 0.14	75.99 ± 0.36	88.83 ± 0.15	87.74 ± 0.19	88.17 ± 0.25	**94.32 ± 0.76**
	Social Evo.	85.70 ± 1.02	93.09 ± 0.45	89.22 ± 0.33	93.67 ± 0.42	84.92 ± 0.03	91.98 ± 0.10	93.49 ± 0.52	86.37 ± 0.26	95.45 ± 0.36	**97.87 ± 0.05**
	UCI	76.67 ± 0.89	60.07 ± 0.24	77.61 ± 0.78	88.35 ± 0.26	86.68 ± 0.41	89.70 ± 0.13	92.38 ± 0.42	91.25 ± 0.18	92.64 ± 0..31	**97.84 ± 0.05**
	Blockchain	92.99 ± 0.08	92.06 ± 0.58	83.09 ± 0.17	95.81 ± 0.16	96.79 ± 0.03	88.71 ± 0.09	96.06 ± 0.35	83.67 ± 0.15	93.67 ± 0.25	**99.07 ± 0.35**
average rank		6.86	8.14	9.00	4.43	5.29	7.14	3.57	5.29	3.86	**1.43**
historical	Wikipedia	61.56 ± 0.02	57.58 ± 0.34	80.87 ± 0.68	76.54 ± 0.87	63.01 ± 0.14	83.05 ± 0.40	61.22 ± 5.41	72.39 ± 0.26	**89.52 ± 0.25**	72.81 ± 0.13
	Reddit	61.77 ± 0.29	61.75 ± 1.32	63.38 ± 0.26	65.87 ± 0.14	65.68 ± 1.04	64.20 ± 0.65	61.24 ± 0.36	63.09 ± 0.12	**81.14 ± 0.19**	62.52 ± 0.25
	Mooc	61.83 ± 1.96	65.57 ± 0.23	68.14 ± 0.52	75.75 ± 0.23	**87.97 ± 0.51**	71.17 ± 0.69	77.58 ± 0.36	79.13 ± 0.33	73.31 ± 0.23	79.64 ± 0.63
	Enron	65.68 ± 0.72	63.68 ± 0.34	54.33 ± 0.59	61.44 ± 0.18	**88.19 ± 0.58**	70.61 ± 0.02	71.48 ± 0.26	67.38 ± 0.20	76.37 ± 0.23	83.33 ± 0.26
	Social Evo.	81.24 ± 1.18	85.46 ± 0.42	84.86 ± 1.07	91.63 ± 0.31	83.19 ± 0.63	89.92 ± 0.62	93.64 ± 0.15	91.26 ± 0.21	91.05 ± 0.14	**99.52 ± 0.05**
	UCI	61.78 ± 1.98	52.43 ± 0.24	59.27 ± 0.77	64.07 ± 0.06	**89.39 ± 0.76**	75.94 ± 0.08	63.29 ± 1.94	70.51 ± 1.06	69.14 ± 0.21	64.94 ± 0.33
	Blockchain	76.49 ± 0.67	83.42 ± 0.14	81.15 ± 0.36	73.47 ± 5.24	68.51 ± 2.07	84.50 ± 0.81	70.88 ± 0.34	75.47 ± 0.17	84.15 ± 0.34	**89.37 ± 0.34**
average rank		8.14	8.14	6.86	5.29	4.57	4.00	6.43	5.00	**3.29**	**3.29**
inductive	Wikipedia	61.55 ± 0.03	55.34 ± 0.24	80.87 ± 0.68	76.54 ± 0.87	63.05 ± 0.24	83.04 ± 0.40	67.48 ± 1.43	72.35 ± 0.36	**88.46 ± 0.31**	72.85 ± 0.85
	Reddit	61.77 ± 0.29	58.24 ± 1.42	63.38 ± 0.26	64.76 ± 0.14	64.18 ± 0.42	64.19 ± 0.65	63.80 ± 0.25	65.06 ± 0.10	**83.24 ± 0.43**	65.75 ± 0.75
	Mooc	61.83 ± 0.19	64.84 ± 0.86	67.90 ± 0.40	75.75 ± 0.23	71.49 ± 0.14	71.18 ± 0.68	**77.79 ± 0.15**	75.83 ± 0.74	75.49 ± 0.21	77.64 ± 0.64
	Enron	65.68 ± 0.72	63.46 ± 0.64	54.35 ± 0.58	61.45 ± 0.17	64.04 ± 0.57	70.63 ± 0.03	71.68 ± 0.52	68.19 ± 0.27	77.42 ± 0.21	**80.68 ± 0.89**
	Social Evo.	81.24 ± 1.18	88.46 ± 2.58	84.86 ± 1.07	91.63 ± 0.14	84.59 ± 0.28	89.92 ± 0.62	95.74 ± 0.36	91.76 ± 0.17	91.22 ± 0.19	**98.66 ± 0.30**
	UCI	61.78 ± 1.97	53.57 ± 0.63	59.28 ± 0.78	64.08 ± 0.06	55.79 ± 0.16	**75.98 ± 0.08**	65.92 ± 0.54	68.41 ± 0.17	68.58 ± 0.24	65.49 ± 0.06
	Blockchain	76.49 ± 0.67	83.41 ± 0.15	81.15 ± 0.35	73.47 ± 5.25	68.51 ± 2.07	84.49 ± 0.81	70.88 ± 0.53	74.37 ± 0.19	85.84 ± 0.41	**86.88 ± 0.53**
average rank		8.14	8.29	7.14	5.57	7.86	4.00	4.71	4.29	2.57	2.43

Strategy: This strategy randomly draws negative edges from almost all possible node pairs in the graph. 2) Historical Negative Sampling Strategy: Here, negative samples are sampled from the set of edges observed at previous timestamps but not present in the current step. 3) Inductive Negative Sampling Strategy: Negative samples are drawn from unseen edges that were not encountered during the training phase. It can be observed that ComCo consistently outperforms all baseline models across various datasets. Specifically, in transductive learning tasks, ComCo achieves average rankings of 2.1 for the transductive learning and 2.04 for inductive learning. This is attributed to: (i) the memory update module maintaining a memory for each node, which integrates all historical information of the node at different scales, thus allowing the use of longer-term node history for downstream tasks; (ii) information from various structural scales captures distinct characteristics of the graph, thereby complementing each other.

Previous studies highlight two essential types of information for this task: temporal information, which captures the network's dynamic evolution over time, and structural information [45, 46], which indicates that the network's topology plays a crucial role in predicting future link formations [47, 48, 49]. However, most existing methods, including those based on Recurrent Neural Networks (RNNs) [23, 24], Self-Attention Mechanisms (SAMs) [22], and mixture of RNN-SAM approaches [7], treat structural and temporal information independently, often overlooking their interdependencies. This limitation results in suboptimal performance. While recent advancements [11, 9] attempt to capture the relationship between structural and temporal information, they either focus solely on modeling the temporal dynamics of a single structural scale or treat different structural scales separately, neglecting the complex interactions between the temporal dynamics of these scales. Consequently, baseline models fail to achieve optimal performance. Our proposed approach effectively addresses these limitations by leveraging a Transformer encoder to establish interconnections across different structural scales and integrating structural and temporal information through the memory update module, leading to improved prediction accuracy and model robustness.

We present the results for dynamic node classification in Table 7 on the Wikipedia and Reddit datasets that have dynamic labels. It can be observed from the table that ComCo achieves better results than most baselines on the Wikipedia and Reddit datasets.

5.3. Ablation Study (RQ2)

We validate the effectiveness of various components in ComCo by individually removing these modules, and we compare it with the following variations.

- $\text{ComCo}_{-w/o-CoE}$ is a variant of ComCo that removes the multi-scale co-evolution module.
- $\text{ComCo}_{-w/o-GraR}$ is a variant of ComCo that removes the dynamic graph representation.

Table 7
ROC-AUC for dynamic node classification.

Methods	Wikipedia	Reddit	Avg.Rank
JODIE	85.90 ± 0.94	60.91 ± 0.92	8.5
DyRep	86.58 ± 0.31	64.94 ± 1.59	6
TGAT	75.20 ± 0.19	65.69 ± 0.62	8
TGN	85.87 ± 0.24	63.54 ± 0.63	7
CAWN	87.32 ± 0.45	69.84 ± 0.64	3.5
GraphMixer	82.05 ± 0.32	63.24 ± 0.41	9
DygFormer	**87.69 ± 0.92**	69.92 ± 1.44	2
FreeDyg	85.42 ± 0.31	67.47 ± 0.54	6.5
CorDGT	86.92 ± 0.14	69.96 ± 0.19	3
Ours	87.49 ± 0.63	**70.04 ± 0.24**	**1.5**

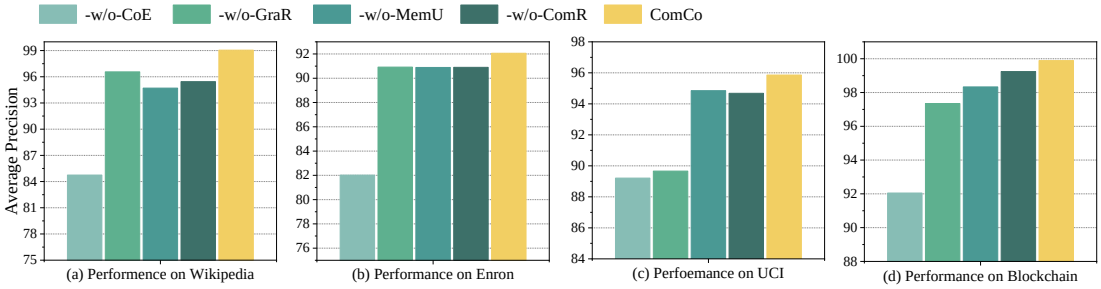


Figure 5: Ablation study results.

- ComCo$_{-w/o-MemU}$ is a variant of ComCo that removes the memory update module.
- ComCo$_{-w/o-ComR}$ is a variant of ComCo that removes the dynamic community representation.

The performance of different ComCo variants is evaluated on four datasets—Wikipedia, Enron, UCI, and Blockchain—as shown in Fig. 5.

The effectiveness of Multi-Scale Representation. ComCo fuses information from three structural scales: individual nodes, communities, and the entire graph. To validate its effectiveness, we conduct an experiment within the Multi-scale Co-Evolution framework where we remove the community and graph representations, relying solely on self-attention for node representation. The results demonstrate that the removal of graph and community representations diminishes the model's capability to express both global and local structures.

The effectiveness of the Memory Update Module. To validate the effectiveness of our memory update module, which integrates nodes' historical information, we concatenate the representations of different structural scales and directly apply them to downstream tasks. As shown in Fig. 5, removing the memory update module may hinder the model's effective use of historical information, compromising its ability to capture temporal dependencies in dynamic graphs.

The effectiveness of the Co-evolution Module. The Transformer-based encoder is designed to capture the intrinsic correlations between different structural scales. To verify its effectiveness, we remove the co-evolution module and directly concatenate representations from different scales. The experimental results demonstrate a significant decline in the model's performance, which validates our primary motivation that capturing the underlying correlations of different structural scales is critical for downstream tasks.

5.4. What Can be Reflected from the Evolution of Community Representations? (RQ3)

Individuals with similar preferences or interests tend to form groups over time. The dynamics of the graph can significantly impact community structure and node representations. To analyze these changes, we employ T-SNE[50] for projecting analysis on the Enron dataset.

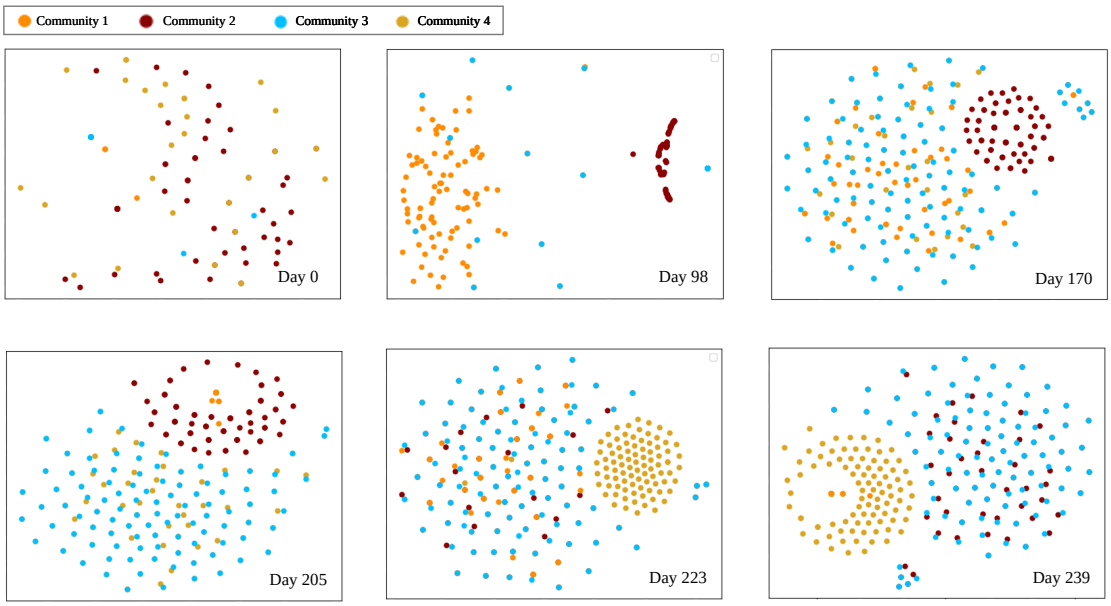


Figure 6: Illustration of the appearance, development, and disappearance of communities in the Enron Dataset. Each point represents a node in a reduced-dimensional space, with different colors indicating different communities.

We utilize four communities to track the temporal and spatial evolution of community members. Each community represents a trending topic, with members clustered based on semantic preferences. Within each community, we limit the maximum number of members to 40 and initiate the projection from the first timestamp. We generate a series of snapshots encompassing the first 1200 interactions over 239 days, as illustrated in Fig. 6. Note that due to irregular intervals between timestamps, the time gaps between snapshots may vary significantly.

In the early stages, our community detection algorithm fails to identify any significant groups on Day 0 due to the limited number of interactions. By Day 98, Community 1 (orange scatters) and Community 2 (red scatters) begin to gain popularity, attracting an increasing number of nodes. Then, on Day 170, Community 3 (blue scatters) emerges and gradually expand indicating that more nodes were drawn to this trend. By Day 205, a growing number of nodes are attracted to Community 4 (gold scatters), while the influence of Community 1 (orange scatters) begins to wane. Subsequently, on Days 223 and 239, community 4 (gold scatters) starts to increase in size and stabilize. This may be attributed to the emergence of new trends, which attracts individuals with similar preferences.

Table 8
AP for TGAT and Dygformer with MemU.

Datasets	TGAT			DygFormer			Ours
	Origin	w/MemU	Improve	Origin	w/MemU	Improve	
Wikipedia	94.85	99.96	5.11%	98.49	99.46	1.47%	**99.83**
Enron	70.09	74.77	4.68%	91.34	94.85	3.51%	**95.06**
UCI	79.03	90.95	11.92%	95.81	99.15	3.34%	**98.95**
Blockchain	86.84	94.10	7.29%	98.91	99.33	0.42%	**99.37**

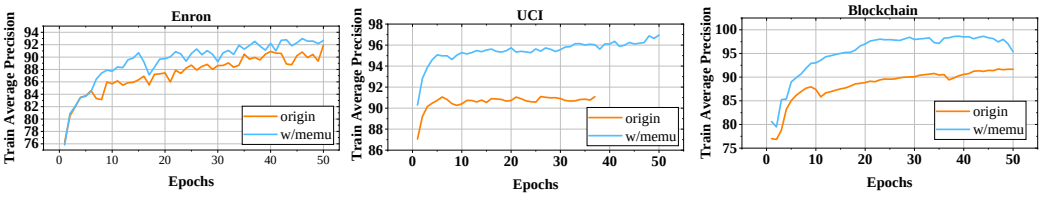
5.5. Generalizability of the Memory Update Module (RQ4)

The memory update module can be easily integrated with dynamic graph learning methods. In this section, we evaluate the enhancement in performance of previous approaches following the incorporation of the memory update module.

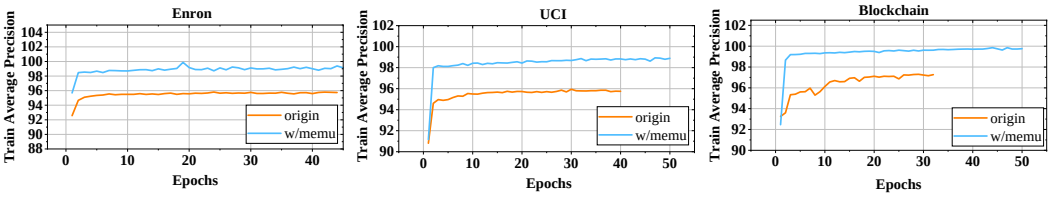
Model performance improvement. We incorporate this module into four conventional methodologies, including TGAT, Dygformer, CAWN, and GraphMixer. The outcomes of the link prediction experiments are reported in Table

Table 9
AP for CAWN and GraphMixer with MemU.

Datasets	CAWN			GraphMixer			Ours
	Origin	w/MemU	Improve	Origin	w/MemU	Improve	
Wikipedia	97.75	99.85	2.10%	96.84	**99.96**	3.12%	99.83
Enron	87.72	95.48	7.76%	82.03	91.02	8.99%	**95.06**
UCI	93.13	98.99	5.87%	93.36	93.24	-0.12%	**98.95**
Blockchain	99.23	99.46	0.47%	97.35	98.88	-1.53%	**99.37**

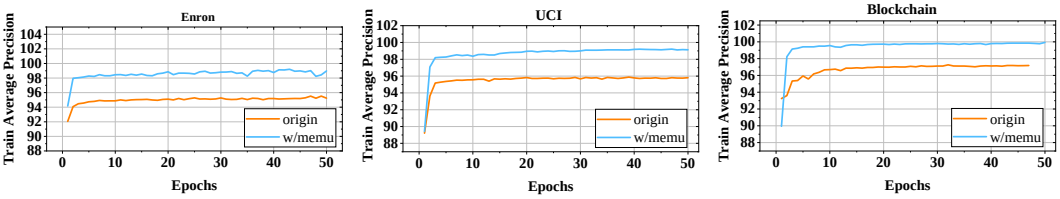


(a) Comparison of training accuracy on TGAT.

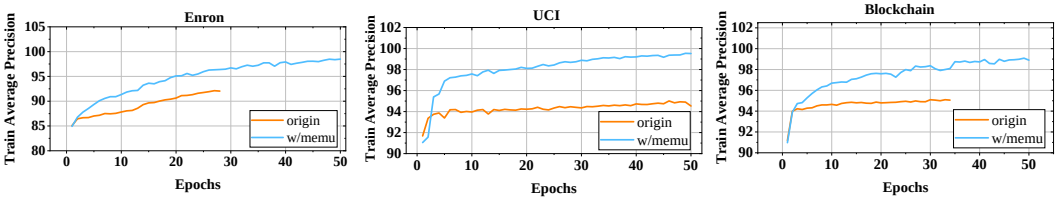


(b) Comparison of training accuracy on DygFormer.

Figure 7: Comparison on the training set average precision for the first 50 training epochs.



(a) Comparison of training accuracy on CAWN.



(b) Comparison of training accuracy on GraphMixer.

Figure 8: Comparison on the training set average precision for the first 50 training epochs.

8 and 9. Notably, the integration of the memory update module (MemU) has led to a significant average performance enhancement. Specifically, TGAT and DygFormer show respective improvements of 7.25% and 2.91%, while CAWN and GraphMixer exhibit enhancements of 4.05% and 2.59% across four datasets.

Higher training accuracy. To better understand the performance of the memory update module, we closely examine the gap in accuracy during the training process. As shown in Fig. 7 and 8, it can be observed that using the memory update module allows the model to achieve a higher average accuracy.

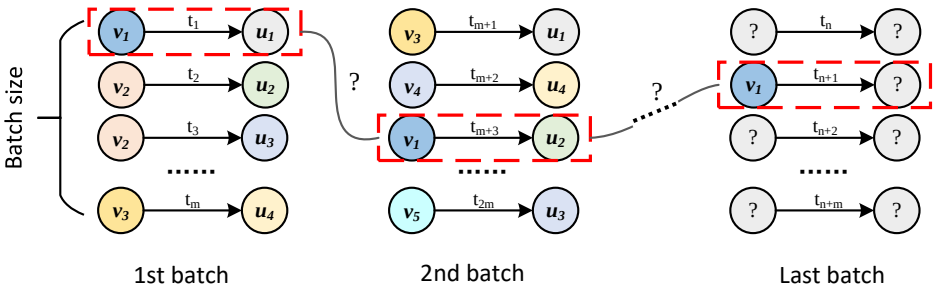
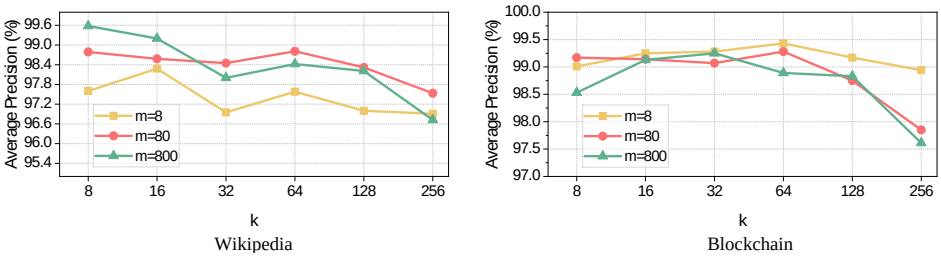
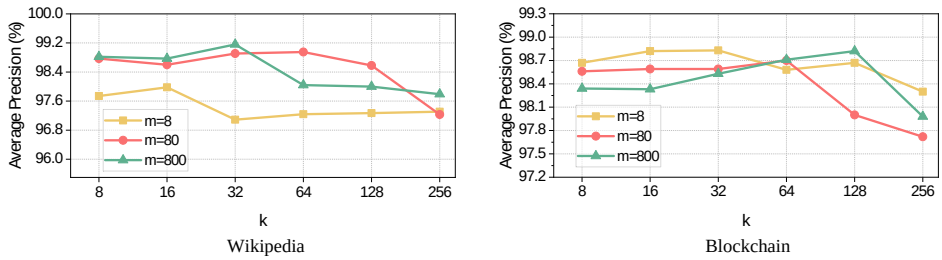


Figure 9: Previous works treated each batch as independent data for training, failing to capture the evolutionary patterns of nodes over time.



(a) Transductive performance of varying parameters k and m.



(b) Inductive performance of varying parameters k and m.

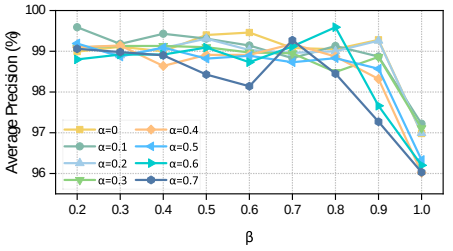
Figure 10: Comparison of varying parameters k and m on the Wikipedia and Blockchain datasets.

The reason why the MemU can work lies in that it effectively addresses the limitations of current dataset processing approaches, particularly the practice of splitting data into separate batches. This splitting fails to capture the ongoing and historical connections of nodes across batches. For example, as illustrated in Fig. 9, node v_1 is active across multiple batches, but if there is a significant time gap between the last batch and the first two, the model struggles to link the interactions of the same entity across different batches as part of its historical behavior during training. MemU overcomes this by maintaining a dedicated memory for each node, aggregating historical information at different scales, and enabling the model to fully utilize interaction information at various timestamps.

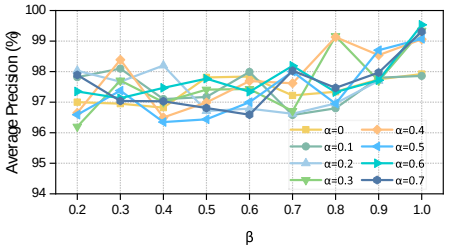
5.6. Influence of Hyperparameters (RQ5)

In this section, we investigate the impact of different parameters on model performance. We conduct link prediction experiments on different datasets and report average precision values.

Impact of parameters k and m. Parameter k determines the maximum number of members. As illustrated in Fig. 10, when the value of k exceeds 128, the model's performance declines significantly. Although more neighbors expand the community range, they also introduce noisy nodes, reducing the expressive power of the community representation. Moreover, excessive community members can lead to high computational costs. We recommend setting k between 32 and 64 for an optimal balance. For parameter m, a larger value detects more communities. On the Wikipedia dataset, smaller m values result in a decline in model performance. Too few communities may not adequately capture network

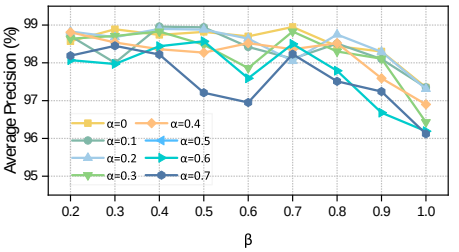


Wikipedia

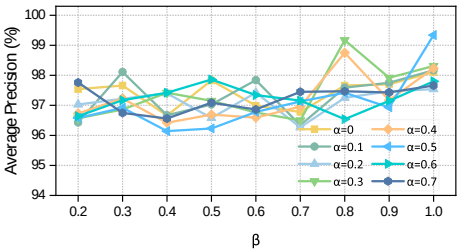


Blockchain

(a) Transductive performance of varying parameters α and β.



Wikipedia



Blockchain

(b) Inductive performance of varying parameters α and β.

Figure 11: Comparison of varying parameters α and β on the Wikipedia and Blockchain datasets.

preferences, while too many can lead to less representative communities. However, the Blockchain dataset is not sensitive to m value. In practice, we suggest setting m to 80 for optimal performance.

Impact of parameter β and α. A smaller β value limits community's perception to a more localized neighborhood, whereas a larger β value expands its perception to a broader neighborhood. As illustrated in Fig. 11, smaller β value lead to better performance on Wikipedia. However, increasing the β value may incorporate less recently community members, which may incorporate noisy neighbors. In contrast, a large β value can lead to better performance on Blockchain, which has diverse interaction patterns. A larger β value, which encompasses a broader neighborhood and captures long-term interactions, proves to be more effective on such diverse graphs. Additionally, a large α value favors 1-hop community neighbors, while a small α value can perceive community neighbors from multiple hops away. Compared to parameter β, ComCo is less sensitive to the changes of parameter α.

5.7. Performance on Noisy or Incomplete Data

ComCo integrates information across multiple structural scales, demonstrating robustness against noisy data. To assess the robustness of our proposed method under noise distribution shifts, we design scenarios involving noisy nodes, missing edges, and community perturbations using the Wikipedia and UCI datasets. In our synthetic experiments, we employ five stochastic transformation functions—DropNode, DropEdge, NoisyEdgeFeat, TimeShift, and CommunityPerturbation—to facilitate evaluation. Since both datasets lack node features, we introduce noise only to the edge features. For the UCI dataset, which does not contain edge features, we report results based on the original data.

Each transformation function includes one or more hyperparameters to control the degree of transformation. For instance, in edge removal, the probability parameter follows a Bernoulli distribution. These transformations introduce varying levels of randomness, ensuring diversity in the generated graph structures. Specifically, for the CommunityPerturbation function, we apply perturbations to both the community fitness score and the community strength score. Since other baselines do not incorporate community detection algorithms, we report only the results for ComCo. The overall results are summarized in Table 10 and 11, and we have the following observations:

1. ComCo demonstrates robustness against noisy data, while the baseline models fail. Specifically, compared to the best-performing baseline for each dataset, ComCo achieves improvements of 2.54% and 3.13% in AP and ROC-AUC, respectively. This improvement may be attributed to ComCo's incorporation of three levels of representation: microscopic, mesoscopic, and macroscopic. At higher structural levels, information becomes more condensed and less

noisy, making the temporal dynamics of these scales more predictable than those at lower levels.

2. Most baseline models consistently yield stable predictions under the impact of temporal shifts. This stability arises because, compared to interaction timing, future interactions between two nodes are more strongly influenced by their historical interactions. Specifically, nodes with a greater number of shared historical neighbors are more likely to interact in the future. Consequently, if node interactions do not exhibit a clear periodicity or temporal pattern, link prediction performance remains largely unaffected by temporal shifts.

3. Compared to NoiseEdgeFeat, most baselines exhibit greater sensitivity to perturbations in topological structure (e.g., DropEdge and DropNode). Specifically, on the Wikipedia dataset, NoisyEdgeFeat causes an average performance decrease of 4.43%, while DropEdge and DropNode result in average decreases of 5.66%. This phenomenon can be attributed to the presence of noise in edge features, which leads models to reduce their reliance on these features and instead prioritize more robust and stable structural cues for predictions. Interestingly, community perturbations have a minimal impact on ComCo's performance, possibly because jointly modeling the temporal dynamics of different structural scales allows information across scales to complement and refine predictions.

Table 10
Transductive performance under different noisy shifts.

	Metric	Approach	DropEdge	DropNode	TimeShift	NoisyEdgeFeat	CommunityPurb	Origin
Wikipedia	AP	TGAT	92.81	85.06	84.23	82.02	-	94.85
		GraphMixer	94.21	88.19	86.49	90.64	-	96.84
		DygFormer	91.39	90.98	99.32	99.02	-	98.49
		FreeDyg	**98.22**	91.32	99.22	**99.23**	-	98.74
		ComCo	98.13	**95.82**	**99.39**	98.80	99.41	**99.83**
	ROC-AUC	TGAT	92.08	85.85	77.65	80.05	-	93.95
		GraphMixer	94.05	87.19	83.43	88.42	-	96.49
		DygFormer	91.66	92.88	99.23	**98.86**	-	98.91
		FreeDyg	**99.13**	93.40	99.13	98.21	-	98.62
		ComCo	98.58	**95.15**	**99.32**	98.66	98.38	**99.79**
UCI	AP	TGAT	74.63	82.72	82.02	79.03	-	79.03
		GraphMixer	77.93	77.93	90.64	93.36	-	93.36
		DygFormer	82.39	86.86	96.09	95.81	-	95.81
		FreeDyg	**95.12**	85.97	95.12	95.25	-	95.25
		ComCo	91.30	**99.95**	**99.12**	**98.95**	98.14	**98.95**
	ROC-AUC	TGAT	74.18	80.78	80.05	79.68	-	79.68
		GraphMixer	79.05	79.05	88.42	91.94	-	91.94
		DygFormer	90.66	87.94	94.89	94.57	-	94.57
		FreeDyg	**93.91**	87.06	93.91	95.94	-	95.94
		ComCo	93.06	**99.37**	**98.84**	**98.69**	98.38	**98.69**

5.8. Comparison with Different Community Detection Methods

To evaluate the effectiveness of ComCo in identifying dynamic community structures and learning community representations, we compare it with three classical algorithms: k-means [51], k-medoids [52], and CDGP [53]. Among these, k-means is a widely used clustering algorithm that partitions data into k clusters based on proximity, while k-medoids follows a similar approach but selects actual data points as cluster centers. CDGP, on the other hand, is a community detection algorithm designed for popularity prediction, capturing the evolution of communities over time.

Since k-means and k-medoids are originally developed for static networks, we apply them to our dynamic framework by clustering all observed nodes and utilizing the LEConv function to select the top-k clusters. The experiment is conducted on the Blockchain dataset, and the results are presented in Table 12.

As shown in Table 12, ComCo consistently outperforms the baseline methods. This superior performance can be attributed to its ability to select community members based on both topological structure and semantic similarity, effectively reducing the inclusion of noisy or less relevant nodes. Furthermore, unlike methods that cluster communities based on all observed nodes, our approach continuously updates community structures as interaction events occur, preserving crucial temporal information.

5.9. Comparison of Training Time for Different Community Member Detection Approaches

We also conduct a comparative analysis of the training time per epoch for various community member detection approaches. Specifically, we examine the time complexity differences between T-PPR (Algorithm 1), which is based on

Table 11
Inductive performance under different noisy shifts.

	Metric	Approach	DropEdge	DropNode	TimeShift	NoisyEdgeFeat	CommunityPurb	Origin
Wikipedia	AP	TGAT	92.02	84.69	81.75	81.76	-	94.54
		GraphMixer	93.29	82.29	86.75	89.83	-	96.31
		DygFormer	91.32	89.88	98.11	**98.51**	-	98.69
		FreeDyg	**98.81**	90.73	98.81	97.89	-	98.68
		ComCo	89.28	**95.10**	**99.04**	98.19	98.69	**99.33**
	ROC-AUC	TGAT	91.48	85.43	77.27	78.77	-	93.68
		GraphMixer	93.02	79.78	84.43	87.96	-	95.49
		DygFormer	90.24	91.62	98.06	**98.44**	-	99.48
		FreeDyg	**98.74**	92.57	98.74	97.83	-	97.22
		ComCo	92.27	**94.35**	**98.84**	98.01	98.43	**99.69**
UCI	AP	TGAT	74.56	82.22	81.76	79.71	-	79.71
		GraphMixer	78.70	78.70	89.83	91.42	-	91.42
		DygFormer	84.42	85.93	94.32	94.34	-	94.34
		FreeDyg	93.91	85.51	93.91	92.23	-	92.23
		ComCo	**95.18**	**99.32**	**97.21**	**97.02**	96.35	**97.02**
	ROC-AUC	TGAT	73.44	79.74	78.77	77.61	-	77.61
		GraphMixer	79.57	79.57	87.96	89.70	-	89.70
		DygFormer	85.56	86.32	92.70	92.38	-	92.38
		FreeDyg	92.23	86.23	92.33	91.25	-	91.25
		ComCo	**97.82**	**99.34**	**98.35**	**97.84**	97.73	**97.84**

Table 12
Performance of community detection methods.

Model	Transductive		Inductive	
	AP	ROC-AUC	AP	ROC-AUC
K-mediods	98.12	98.84	97.05	97.11
K-means	98.11	98.07	97.24	97.36
CDGP	98.87	**99.87**	97.12	98.23
ComCo	**99.37**	99.29	**99.17**	**99.07**

graph diffusion, and T-GNN, a method that utilizes recursive temporal message passing. The results, as illustrated in Fig 8, indicate that as the maximum number of community members increases, the T-GNN approach incurs significantly longer training times compared to T-PPR.

This discrepancy arises because T-PPR employs a single-scan algorithm to identify the most influential temporal neighbors without requiring recursive aggregation, thereby avoiding the exponential growth in computation graph size that affects T-GNN. Specifically, in T-GNN, let k denote the maximum number of temporal neighbors sampled for each temporal node, and let L represent the number of layers. At the l-th layer, the number of nodes is $O(k^{L-l})$, where $1 \leq l \leq L$. Furthermore, T-GNN aggregates the features of up to k temporal neighbors to compute the representation of a temporal node. Consequently, the time complexity of an L-layer T-GNN is given by: $O\left(\sum_{l=1}^{L} k \cdot k^{(L-l)}\right) = O(k^L)$. In contrast, as discussed in Section 4.3, the time complexity for answering a top-k T-PPR query is $O(kd + k \log k)$. In summary, compared to traditional temporal message-passing methods, the T-PPR-based approach achieves significantly lower time complexity, making it more efficient for large-scale community detection tasks.

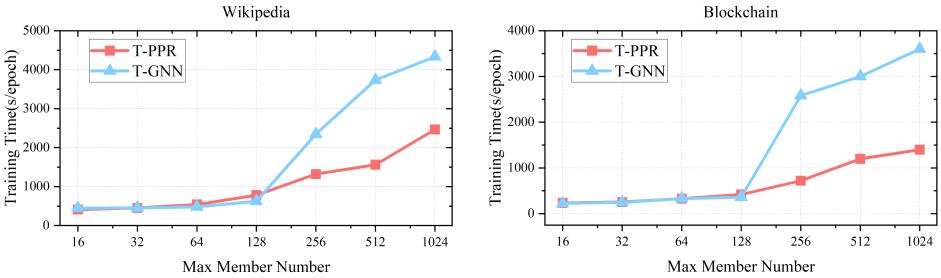


Figure 12: Comparison of training time per epoch between T-PPR and T-GNN.

6. Conclusion and Discussion

In this paper, we present ComCo, a Community-based multi-scale Co-evolving network, which is designed to fuse the microscopic, mesoscopic and macroscopic structure information by learning the intrinsic relationships across these structural scales. To capture the long-term evolution of individual preferences and features, our approach leverages a dynamic and general representation for each node, which is updated as long as a related interaction is observed. To bridge the connection between microscopic structures and the overall macroscopic network, we propose a novel community detection mechanism to dramatically group nodes with similar preferences. Moreover, we introduce a multi-scale co-evolution mechanism to explicitly characterize the complex relationship among entities, communities, and the network as a whole. Extensive experiments on seven datasets demonstrate the superiority of multi-scale structures in various downstream tasks, a further ablation study shows the effectiveness of our designs. Recent studies highlight the potential of contrastive learning on dynamic graphs, which suggests a future exploration to apply contrastive learning across different structural scales in dynamic graphs.

Limitations The performance of ComCo is partially dependent on the effectiveness of the memory update module (MemU), as discussed in Section 5.5. However, we observe that MemU can negatively impact performance in both the historical (hist) and inductive (ind) strategies, particularly by converting previously positive links into negative ones. Here we attribute this issue to the fact that, when an interaction occurs, MemU updates both the source and target nodes, which may reinforce positive links. As a result, MemU can sometimes be sensitive to different negative sampling strategies, especially when its intended behavior is disrupted. Moreover, RNN-based models face challenges such as vanishing/exploding gradients and high computational costs due to the use of recurrent neural networks [54, 55, 56, 57]. Randomly selecting a subset of nodes for updates and updating only one of the interacting nodes may help mitigate these issues.

Future work While our current approach effectively models edge addition events in weighted dynamic graphs, several promising research directions remain open for further improvement. One key avenue for future exploration is enhancing the accuracy of **edge weight prediction**. Unlike unweighted graphs, where the primary concern is determining the presence or absence of an edge, weighted graphs require the model to not only predict connectivity but also estimate the strength of relationships. To achieve this, two important modifications can be introduced: incorporating edge weights into the encoder layer and designing weight-aware message-passing functions that assign varying levels of influence to neighbors based on edge strength. Additionally, transitioning from cross-entropy loss to mean squared error (MSE) loss could further refine the model's ability to capture nuanced weight variations.

Another critical challenge in dynamic graphs is handling **edge deletion events**, an aspect that remains under-explored in our current work. Many real-world networks experience frequent edge deletions, and failing to account for these changes can lead to outdated or misleading graph representations. To address this, an edge-removal-aware propagation mechanism could be developed by introducing an adaptive attention mechanism that dynamically recalibrates node embeddings in response to edge removals.

Ethical Approval

Not applicable.

Consent to participate

There is the consent of all authors.

Human and Animal Ethics

Not applicable.

Consent for publication

There is the consent of all authors.

Data availability

The datasets used in the experiments are publicly available in the online repository.

Declaration of competing interest

The authors declare that there is no competing interests.

CRediT authorship contribution statement

Chao Li: Writing – review & editing, Validation. **Qianyu Song:** Writing – original draft, Methodology, Data curation. **Runshuo Liu:** Writing – review & editing, Resources. **Zhongying Zhao:** Writing – review & editing, Conceptualization. **Qingtian Zeng:** Writing – review & editing, Resources.

Acknowledgments

This research is supported by the National Natural Science Foundation of China (Grant No.62072288), Humanities and Social Science Research Project of the Ministry of Education (Grant No.24YJAZH058), the Natural Science Foundation of Shandong Province (Grant No.ZR2022MF268); ShandongYouth Innovation Team; the Taishan Scholar Program of Shandong Province (Grant No.tsqn202211154)

References

[1] Unai Alvarez-Rodriguez, Federico Battiston, Guilherme Ferraz de Arruda, Yamir Moreno, Matjaž Perc, and Vito Latora. Evolutionary dynamics of higher-order interactions in social networks. *Nature Human Behaviour*, 5:586 – 595, 2020.

[2] Jicun Jiang, Xiaodi Liu, Zengwen Wang, Weiping Ding, and Shitao Zhang. Large group emergency decision-making with bi-directional trust in social networks: A probabilistic hesitant fuzzy integrated cloud approach. *Information Fusion*, 102:102062, 2024.

[3] Jie Guo, Yan Zhou, Peng Zhang, Bin Song, and Chen Chen. Trust-aware recommendation based on heterogeneous multi-relational graphs fusion. *Information Fusion*, 74:87–95, 2021.

[4] Roza Abolghasemi, Enrique Herrera Viedma, Paal Engelstad, Youcef Djenouri, and Anis Yazidi. A graph neural approach for group recommendation system based on pairwise preferences. *Information Fusion*, 107:102343, 2024.

[5] Yi Xu, Liangzhe Han, Tongyu Zhu, Leilei Sun, Bowen Du, and Weifeng Lv. Generic dynamic graph convolutional network for traffic flow forecasting. *Information Fusion*, 100:101946, 2023.

[6] Senthil Murugan Nagarajan, Ganesh Gopal Devarajan, Ramana T.V., Asha Jerlin M., Ali Kashif Bashir, and Yasser D. Al-Otaibi. Adversarial deep learning based dampster–shafer data fusion model for intelligent transportation system. *Information Fusion*, 102:102050, 2024.

[7] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael M. Bronstein. Temporal graph networks for deep learning on dynamic graphs. *CoRR*, abs/2006.10637, 2020.

[8] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. Do we really need complicated model architectures for temporal networks? In *The Eleventh International Conference on Learning Representations*, 2023.

[9] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems*, 36:67686–67700, 2023.

[10] Giang Hoang Nguyen, John Boaz Lee, Ryan A. Rossi, Nesreen Ahmed, Eunyee Koh, and Sungchul Kim. Continuous-time dynamic network embeddings. *Companion Proceedings of the The Web Conference 2018*, 2018.

[11] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. Inductive representation learning in temporal networks via causal anonymous walks. In *International Conference on Learning Representations*, 2021.

[12] Ming Jin, Yuan-Fang Li, and Shirui Pan. Neural temporal walks: Motif-aware representation learning on continuous-time dynamic graphs. In *Neural Information Processing Systems*, 2022.

[13] Guozhen Zhang, Tian Ye, Depeng Jin, and Yong Li. An attentional multi-scale co-evolving model for dynamic link prediction. In *Proceedings of the ACM Web Conference 2023*, pages 429–437, 2023.

[14] William H. Warren, J. Benjamin Falandays, Kei Yoshida, Trenton D. Wirth, and Brian A. Free. Human crowds as social networks: Collective dynamics of consensus and polarization. 19:522–537, 2024.

[15] Mengbin Ye, Lorenzo Zino, Zan Mlakar, Jan Willem Bolderdijk, Hans Risselada, Bob M. Fennis, and Ming Cao. Collective patterns of social diffusion are shaped by individual inertia and trend-seeking. 12, 2021.

[16] Yi-Zi Ning, Xin Liu, Hui-Min Cheng, and Zhongyuan Zhang. Effects of social network structures and behavioral responses on the spread of infectious diseases. *Physica A: Statistical Mechanics and its Applications*, 2020.

[17] Hendrik Nunner, Vincent Buskens, Alexandra Teslya, and Mirjam Kretzschmar. Health behavior homophily can mitigate the spread of infectious diseases in small-world networks. 312:115350–115350, 2022.

[18] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao Schardl, and Charles Leiserson. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 5363–5370, 2020.

[19] Jiaxuan You, Tianyu Du, and Jure Leskovec. Roland: graph learning framework for dynamic graphs. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pages 2358–2366, 2022.

[20] Weilin Cong, Yanhong Wu, Yuandong Tian, Mengting Gu, Yinglong Xia, Chun-cheng Jason Chen, and Mehrdad Mahdavi. Dyformer: A scalable dynamic graph transformer with provable benefits on generalization ability. In *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM)*, pages 442–450. SIAM, 2023.

[21] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. Dysat: Deep neural representation learning on dynamic graphs via self-attention networks. In *Proceedings of the 13th international conference on web search and data mining*, pages 519–527, 2020.

[22] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. Inductive representation learning on temporal graphs. In *International Conference on Learning Representations*, 2020.

[23] Srijan Kumar, Xikun Zhang, and Jure Leskovec. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1269–1278, 2019.

[24] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. Dyrep: Learning representations over dynamic graphs. In *International conference on learning representations*, 2019.

[25] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, et al. Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 international conference on management of data*, pages 2628–2638, 2021.

[26] Ziwei Fan, Zhiwei Liu, Jiawei Zhang, Yun Xiong, Lei Zheng, and Philip S Yu. Continuous-time sequential recommendation with temporal graph collaborative transformer. In *Proceedings of the 30th ACM international conference on information and knowledge management*, pages 433–442, 2021.

[27] Weimin Li, Xiaokang Zhou, Chao Yang, Yuting Fan, Zhao Wang, and Yanxia Liu. Multi-objective optimization algorithm based on characteristics fusion of dynamic social networks for community discovery. *Information Fusion*, 79:110–123, 2022.

[28] Mengyue Liu, Jun Liu, Yixiang Dong, Rui Mao, and Erik Cambria. Interest-driven community detection on attributed heterogeneous information networks. *Information Fusion*, 111:102525, 2024.

[29] Xiaotong Zhang, Han Liu, Qimai Li, and Xiao-Ming Wu. Attributed graph clustering via adaptive graph convolution. *arXiv preprint arXiv:1906.01210*, 2019.

[30] Hongtao Liu, Jiahao Wei, and Tianyi Xu. Community detection based on community perspective and graph convolutional network. *Expert Systems with Applications*, 231:120748, 2023.

[31] Xixi Wu, Yun Xiong, Yao Zhang, Yizhu Jiao, Caihua Shan, Yiheng Sun, Yangyong Zhu, and Philip S Yu. Clare: A semi-supervised community detection algorithm. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pages 2059–2069, 2022.

[32] Qiqi Zhao, Huifang Ma, Lijun Guo, and Zhixin Li. Hierarchical attention network for attributed community detection of joint representation. *Neural Computing and Applications*, 34(7):5587–5601, 2022.

[33] Bang Wang, Xiang Cai, Minghua Xu, and Wei Xiang. A graph-enhanced attention model for community detection in multiplex networks. *Expert Systems with Applications*, 230:120552, 2023.

[34] Ruina Bai, Ruizhang Huang, Luyi Zheng, Yanping Chen, and Yongbin Qin. Structure enhanced deep clustering network via a weighted neighbourhood auto-encoder. *Neural Networks*, 155:144–154, 2022.

[35] Jie Wen, Gehui Xu, Zhanyan Tang, Wei Wang, Lunke Fei, and Yong Xu. Graph regularized and feature aware matrix factorization for robust incomplete multi-view clustering. *IEEE Transactions on Circuits and Systems for Video Technology*, 34(5):3728–3741, 2024.

[36] Chengliang Liu, Jie Wen, Zhihao Wu, Xiaoling Luo, Chao Huang, and Yong Xu. Information recovery-driven deep incomplete multiview clustering network. *IEEE Transactions on Neural Networks and Learning Systems*, 35(11):15442–15452, 2024.

[37] Yiming Li, Yanyan Shen, Lei Chen, and Mingxuan Yuan. Zebra: When temporal graph neural networks meet temporal personalized pagerank. *Proceedings of the VLDB Endowment*, 16(6):1332–1345, 2023.

[38] Songlai Ning, Jiakang Li, and Yonggang Lu. A comprehensive review of community detection in graphs. *arXiv preprint arXiv:2309.11798*, 2023.

[39] Bolian Li, Baoyu Jing, and Hanghang Tong. Graph communal contrastive learning. In *Proceedings of the ACM web conference 2022*, pages 1203–1213, 2022.

[40] Jie Wen, Shijie Deng, Lunke Fei, Zheng Zhang, Bob Zhang, Zhao Zhang, and Yong Xu. Discriminative regression with adaptive graph diffusion. *IEEE Transactions on Neural Networks and Learning Systems*, 35(2):1797–1809, 2024.

[41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[42] Farimah Poursafaei, Shenyang Huang, Kellin Pelrine, and Reihaneh Rabbany. Towards better evaluation for dynamic link prediction. *Advances in Neural Information Processing Systems*, 35:32928–32941, 2022.

[43] Yuxing Tian, Yiyan Qi, and Fan Guo. Freedyg: Frequency enhanced continuous-time dynamic graph model for link prediction. In *International Conference on Learning Representations*, 2024.

[44] Zhe Wang, Sheng Zhou, Jiawei Chen, Zhen Zhang, Binbin Hu, Yan Feng, Chun Chen, and Can Wang. Dynamic graph transformer with correlated spatial-temporal positional encoding. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*, 2025.

[45] Yuanfu Lu, Xiao Wang, Chuan Shi, Philip S Yu, and Yanfang Ye. Temporal network embedding with micro-and macro-dynamics. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 469–478, 2019.

[46] Aynaz Taheri, Kevin Gimpel, and Tanya Berger-Wolf. Learning to represent the evolution of dynamic graphs with recurrent models. In *Companion proceedings of the 2019 world wide web conference*, pages 301–307, 2019.

[47] Lei Cai and Shuiwang Ji. A multi-scale approach for graph link prediction. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 3308–3315, 2020.

[48] Chengbin Hou, Han Zhang, Shan He, and Ke Tang. Glodyne: Global topology preserving dynamic network embedding. *IEEE Transactions on Knowledge and Data Engineering*, 34(10):4826–4837, 2020.

[49] Lekui Zhou, Yang Yang, Xiang Ren, Fei Wu, and Yueting Zhuang. Dynamic network embedding by modeling triadic closure process. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[50] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

[51] Abiodun M. Ikotun, Absalom E. Ezugwu, Laith Abualigah, Belal Abuhaija, and Jia Heming. K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Information Sciences*, 622:178–210, 2023.

[52] Jiangzhou Deng, Junpeng Guo, and Yong Wang. A novel k-medoids clustering recommendation algorithm based on probability distribution for collaborative filtering. *Knowl. Based Syst.*, 175:96–106, 2019.

[53] Shuo Ji, Xiaodong Lu, Mingzhe Liu, Leilei Sun, Chuanren Liu, Bowen Du, and Hui Xiong. Community-based dynamic graph learning for popularity prediction. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 930–940, 2023.

[54] Lu Wang, Xiaofu Chang, Shuang Li, Yunfei Chu, Hui Li, Wei Zhang, Xiaofeng He, Le Song, Jingren Zhou, and Hongxia Yang. Tcl: Transformer-based dynamic graph modelling via contrastive learning. abs/2105.07944, 2021.

[55] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning*, 2012.

[56] Yuhong Luo and Pan Li. Neighborhood-aware scalable temporal network representation learning. *Learning on Graphs Conference*, 2022.

[57] Yao Ma, Ziyi Guo, Zhaocun Ren, Jiliang Tang, and Dawei Yin. Streaming graph neural networks. SIGIR '20, page 719–728, New York, NY, USA, 2020. Association for Computing Machinery.